

# CS-229

- Quiz 1
- Intro Computer Vision
  - Tasks
  - Architectures / backbones / prior knowledge



*Prompt: "A robot is doing taxes, but is in despair because it's too complicated."*

100%

High Score

78%

Low Score

93%

Mean Score

100%

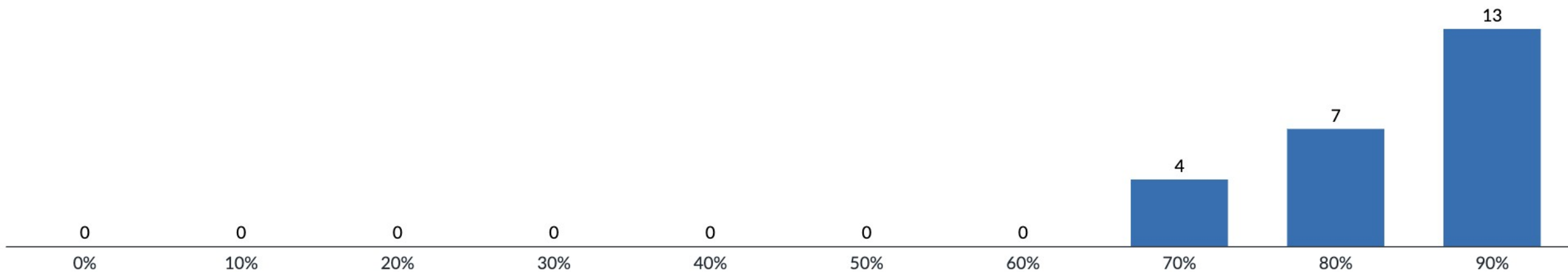
Median Score

06:19:21

Mean Elapsed  
Time



## Score Distribution



Get some scratch paper ready for the next few. ^

Suppose we have three words, "Attention to words", and they have word embeddings,

$x_{\text{Attention}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, x_{\text{to}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, x_{\text{words}} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ . For simplicity, assume that the

key, query and value matrices for each word are all equal,

$k_i = \mathbb{I} x_i, q_i = \mathbb{I} x_i, v_i = \mathbb{I} x_i$ .

We're going to calculate the output of a dot-product attention on this input. (Don't use scaling, so we get  $\text{softmax}(QK^T) V$ , where Q, K, V are matrices that stack the vectors,

like  $Q = \begin{pmatrix} - & \vec{q}_1 & - \\ - & \vec{q}_2 & - \\ - & \vec{q}_3 & - \end{pmatrix}$ ).

First we need the dot product matrix, then we take the softmax. The output of the second position (with "to" as the input) pays the most "attention" (i.e. has highest probability) to which token?

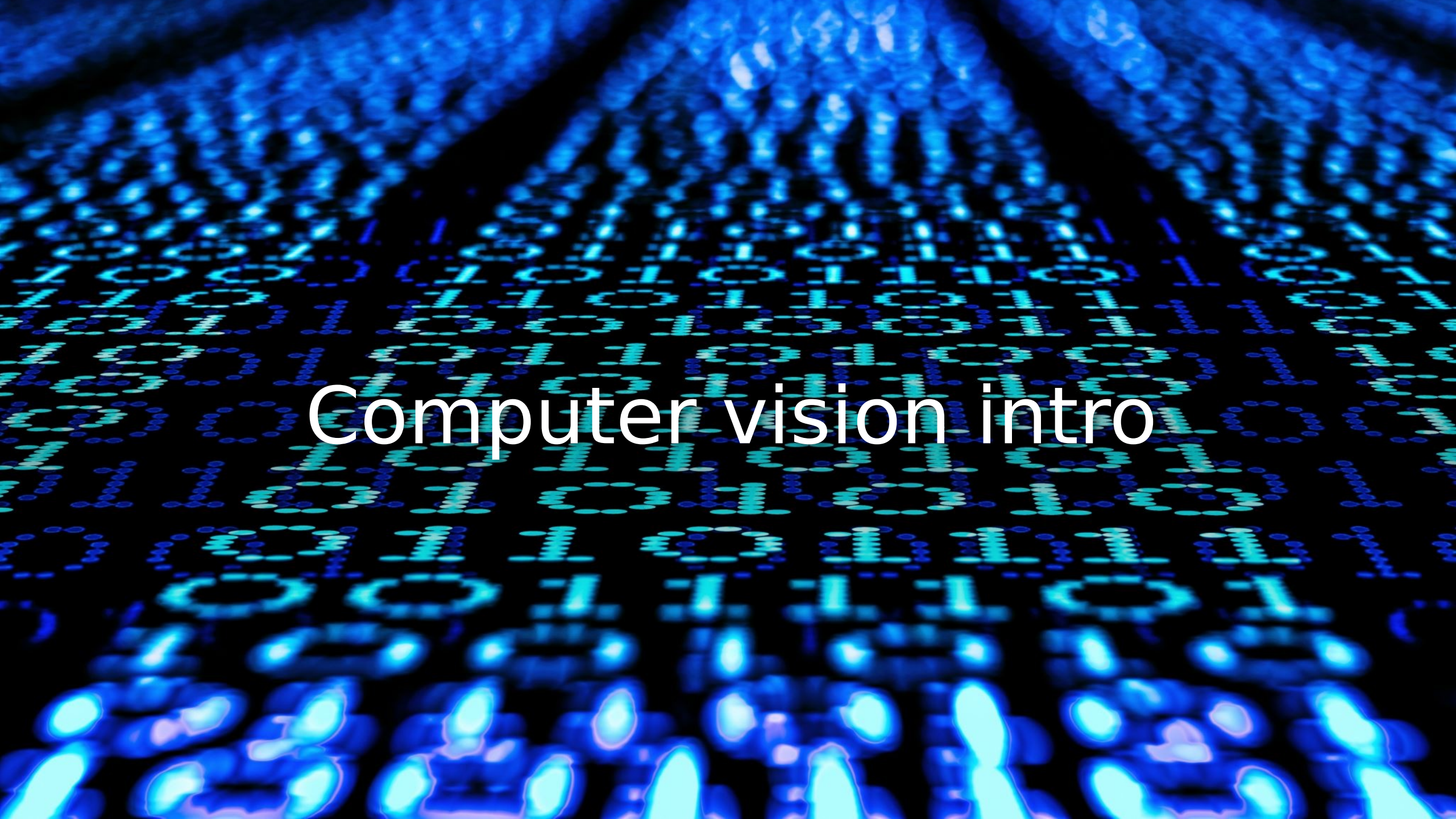
### Answer Frequency Summary

|   | Answer | Respondents | Percentage |
|---|--------|-------------|------------|
| ✓ | words  | 19          | 79%        |
| ✗ | to     | 3           | 13%        |

Continuing what is the output vector in the first position (associated with "Attention")  
( $x_i$  refers to vectors from the previous question.)

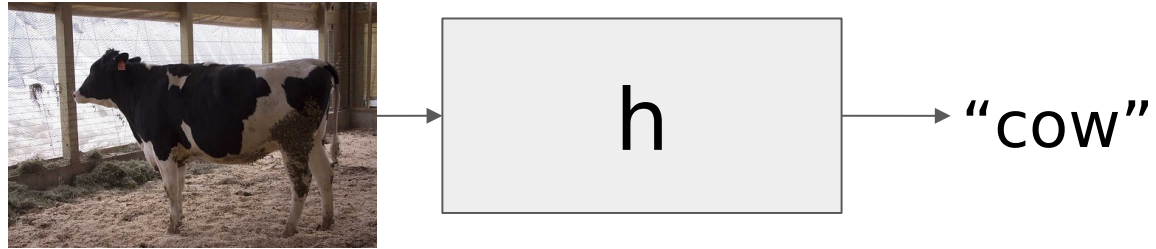
### Answer Frequency Summary

|   | Answer                                                                                        | Respondents | Percentage |
|---|-----------------------------------------------------------------------------------------------|-------------|------------|
| ✗ | $\frac{1}{2} x_{\text{Attention}} + \frac{1}{2} x_{\text{words}}$                             | 3           | 13%        |
| ✓ | $0.42 x_{\text{Attention}} + 0.16 x_{\text{to}} + 0.42 x_{\text{words}}$                      | 18          | 75%        |
| ✗ | $\frac{1}{8} x_{\text{Attention}} + \frac{1}{4} x_{\text{to}} + \frac{5}{8} x_{\text{words}}$ | 0           | 0%         |
| ✗ | $x_{\text{Attention}}$                                                                        | 3           | 13%        |



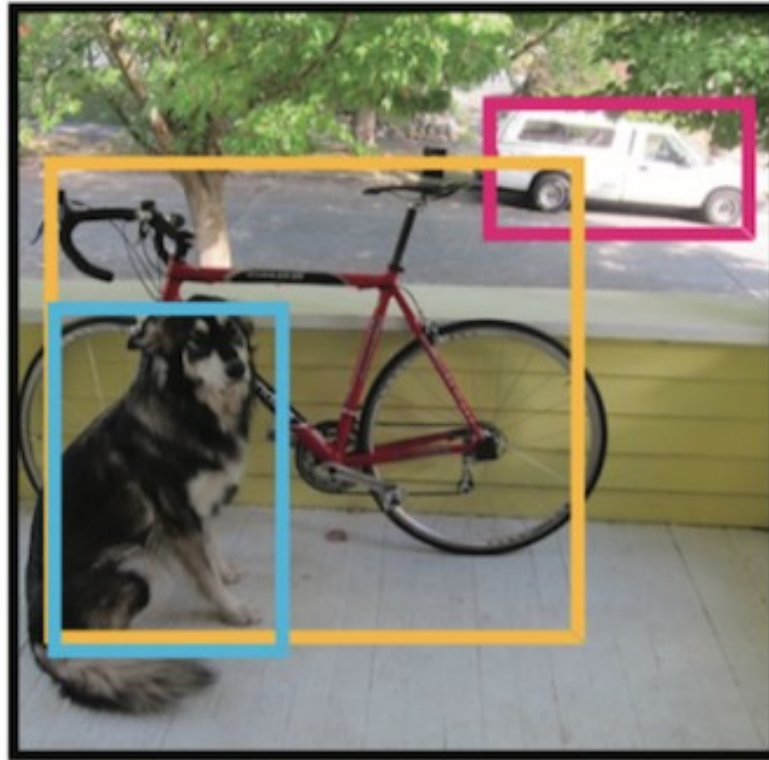
# Computer vision intro

# Image Classification



# Object detection

- Output a bounding box and a category
- Many objects possible per image



(Semantic) Segmentation – assign a category to each *pixel*

[Segment Anything](#)



# *Instance* segmentation

[Segment Anything](#)



# Many other tasks

- Face or pose recognition
- 3D reconstruction or depth mapping
- Video tasks
  - Object tracking
  - Activity recognition
  - Temporal segmentation
- Multimodal (A little on Monday)
  - Image captioning
  - Visual question answering
  - OCR – optical character recognition
- Generative tasks (later in the course)
  - Image restoration
  - Inpainting
  - Super-resolution
  - Style transfer / image synthesis

Architectures  
encode our  
intuition / prior  
knowledge  
about vision

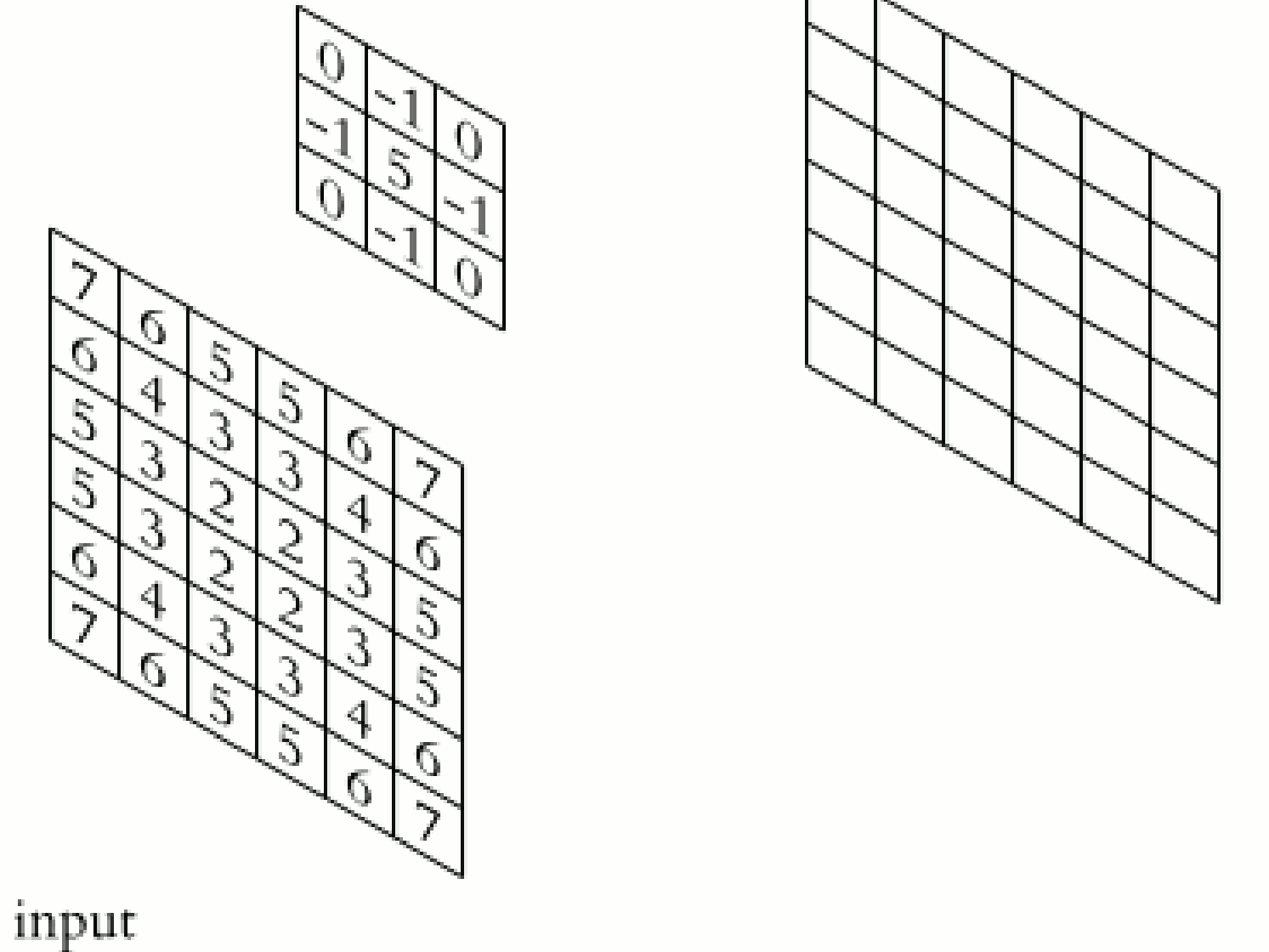
Convolution –  
locality and  
spatial invariance

Pooling –  
translation  
invariance

Attention

# Convolutio

- Convolutional kernel – combines each pixel with info from neighboring pixels (locality)
- Same weights applied spatially everywhere (spatial invariance)
- If there are  $N$  pixels, this is a  $N \rightarrow N$  mapping. How many weights would it generally take?



output

input

# Convolution\* in 1D (invariant/equivariant?)

- Input vector  $\mathbf{x}$ :

$$\mathbf{x} = [x_1, x_2, \dots, x_I]$$

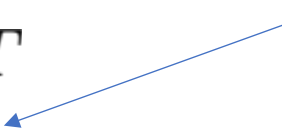
- Output is weighted sum of neighbors:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

- Convolutional **kernel** or **filter**:

$$\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$$

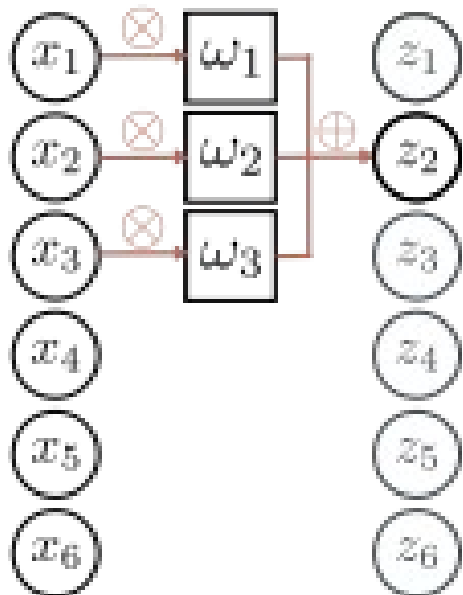
Kernel size = 3



\* different from signal processing

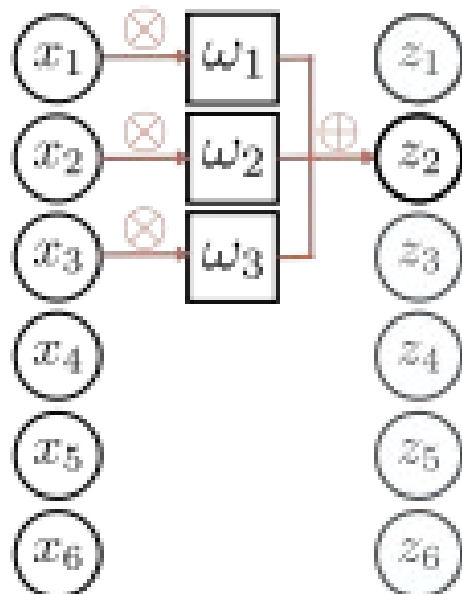
# Convolution with kernel size 3

a)

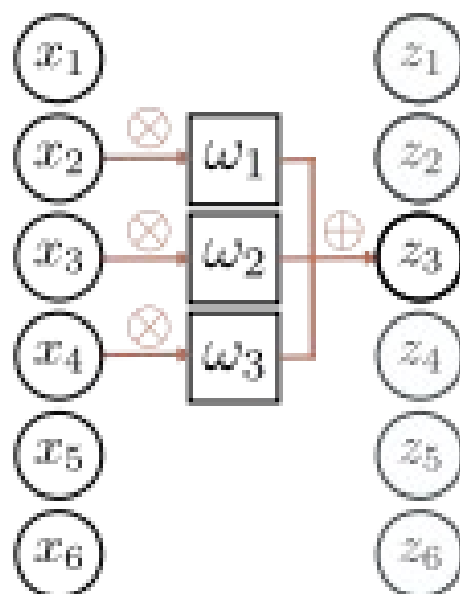


# Convolution with kernel size 3

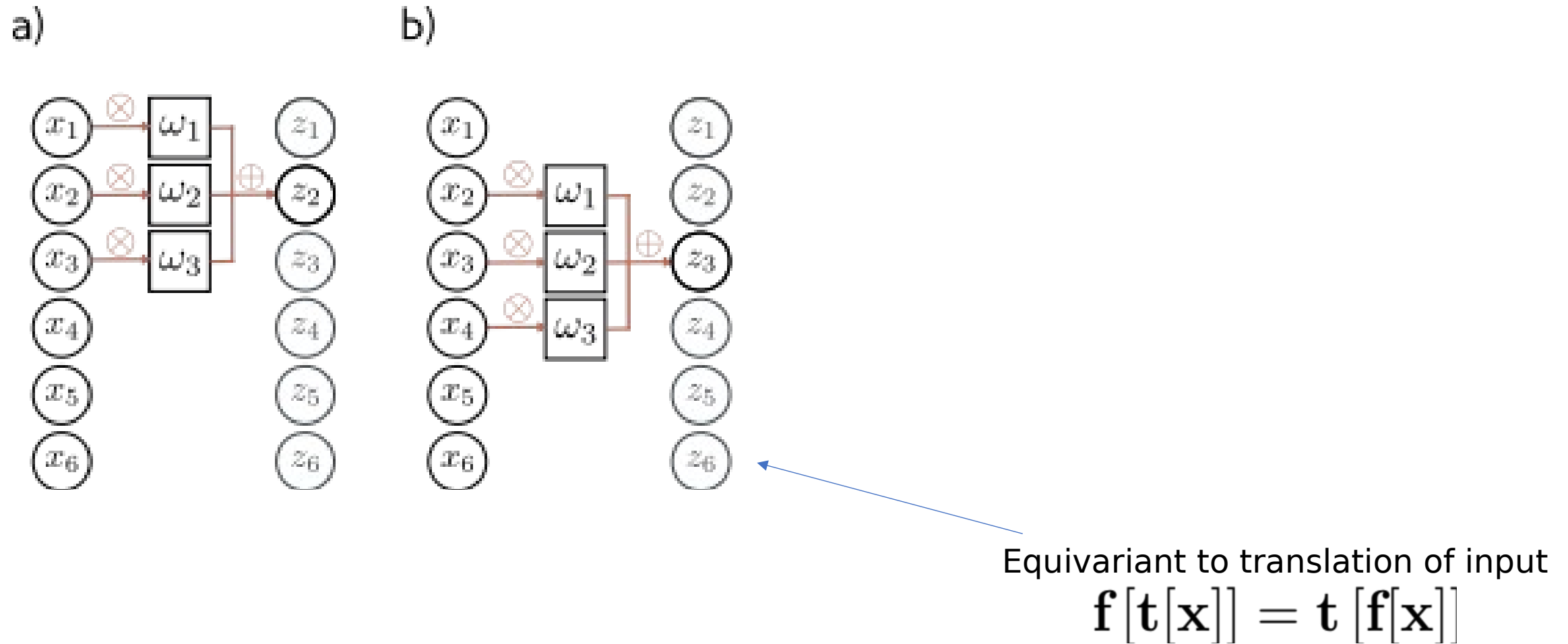
a)



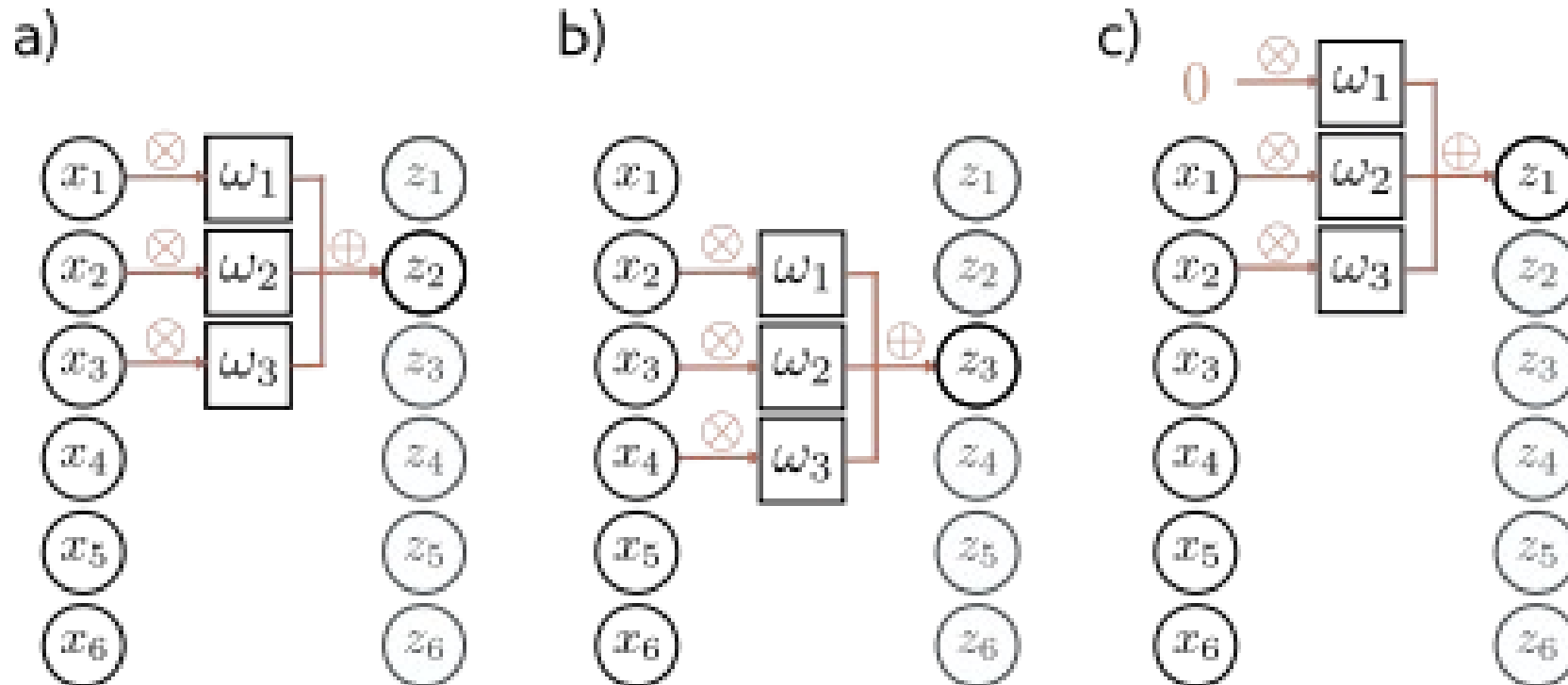
b)



# Convolution with kernel size 3

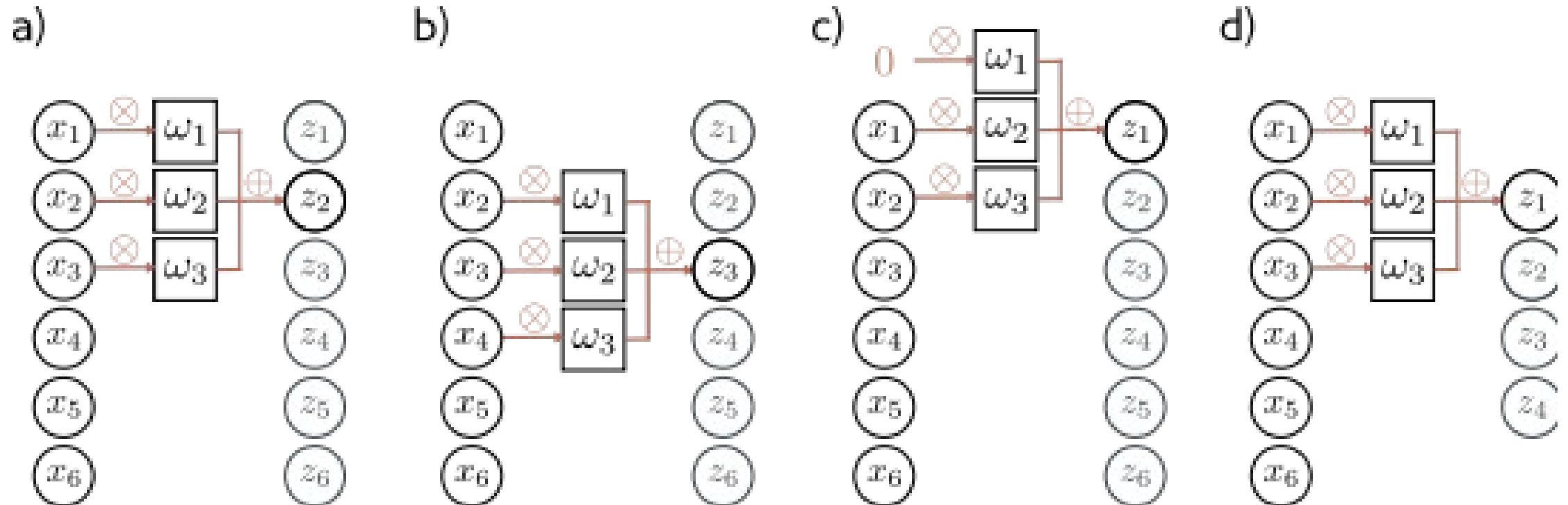


# Zero padding



Treat positions that are beyond end of the input as zero.

# “Valid” convolutions

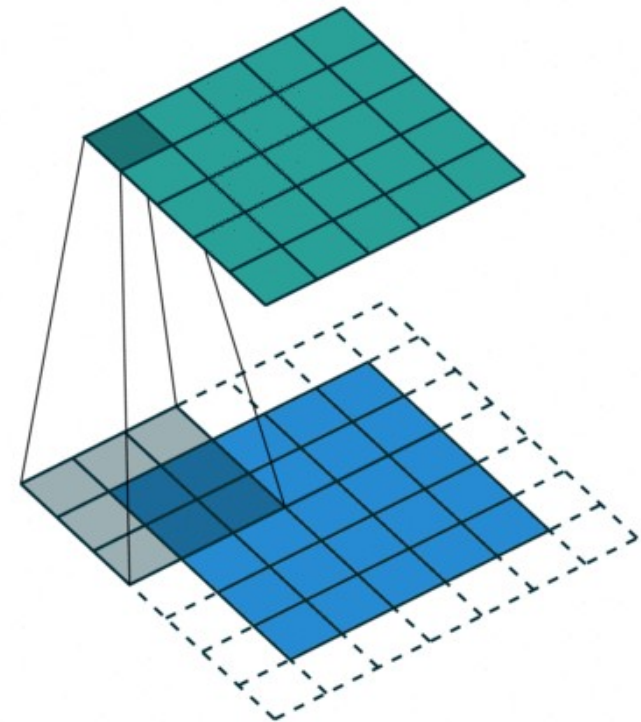


Only process positions where kernel falls in image (smaller output).

# Padding in convolution (2d)

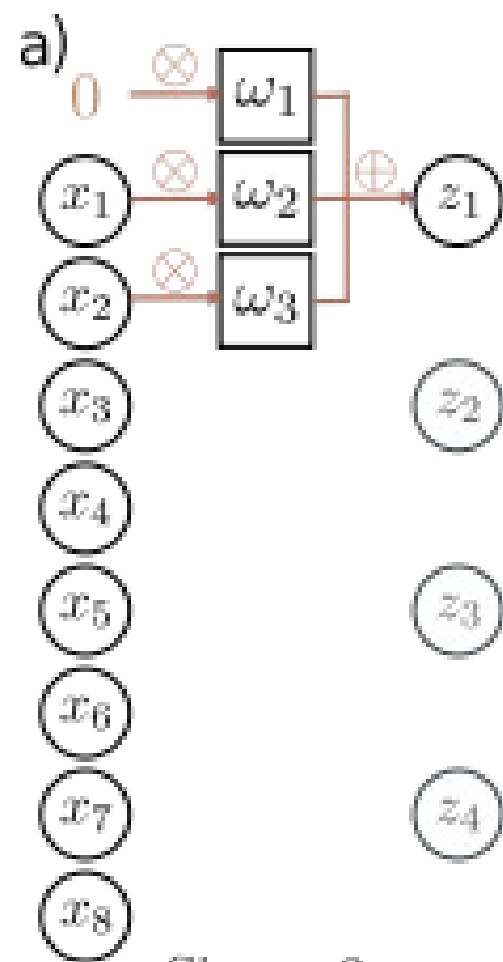
<https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html>

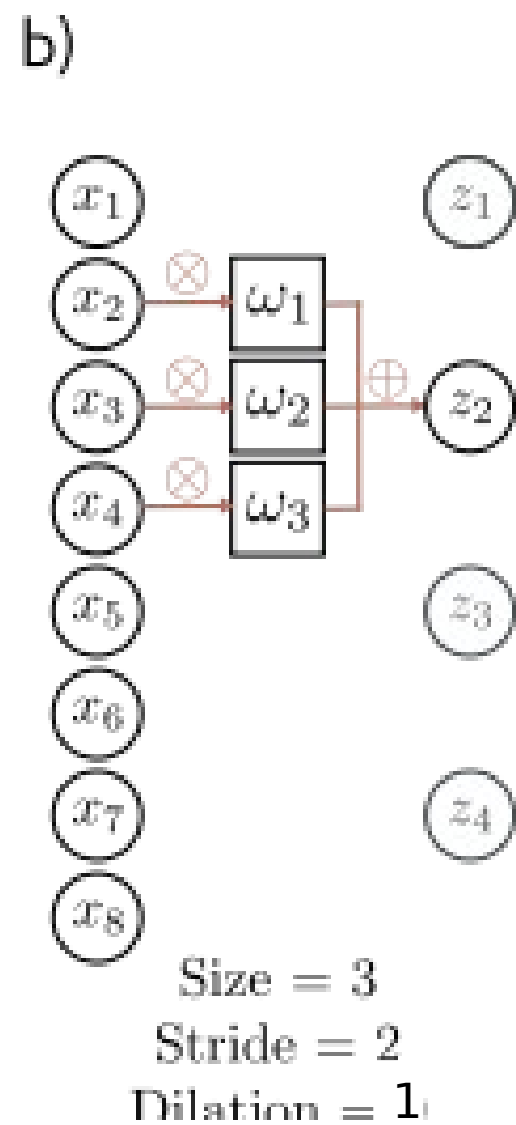
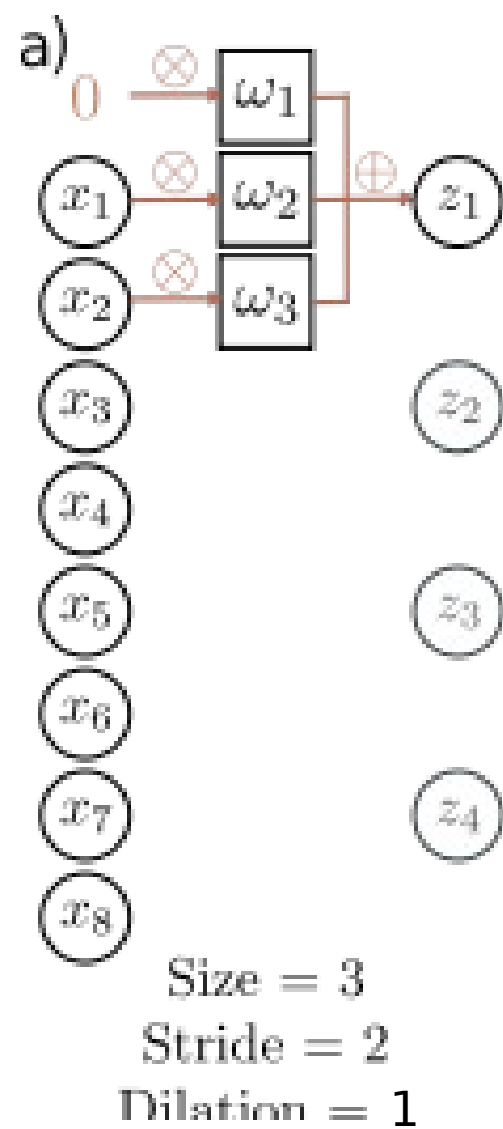
For me visualizations make this easier

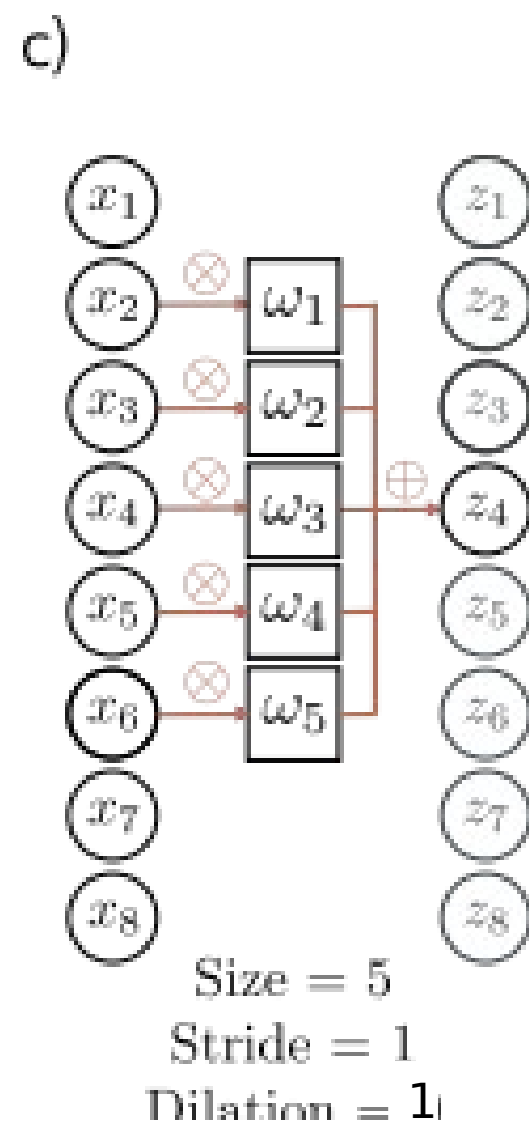
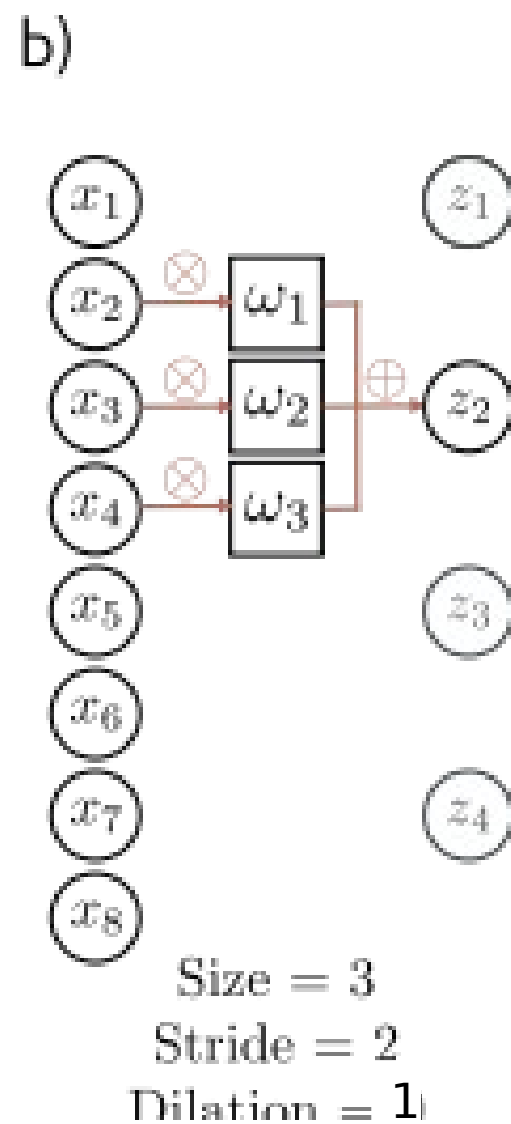
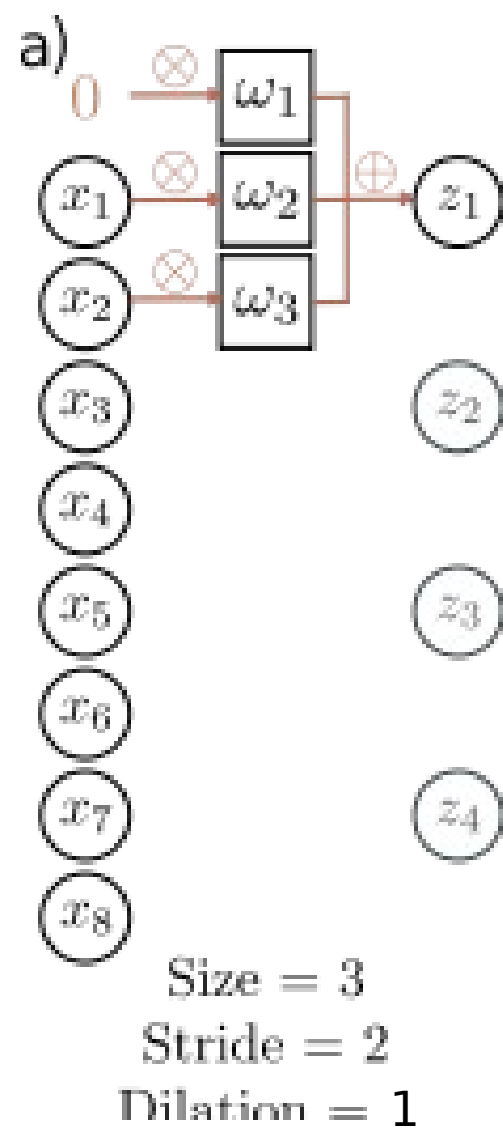


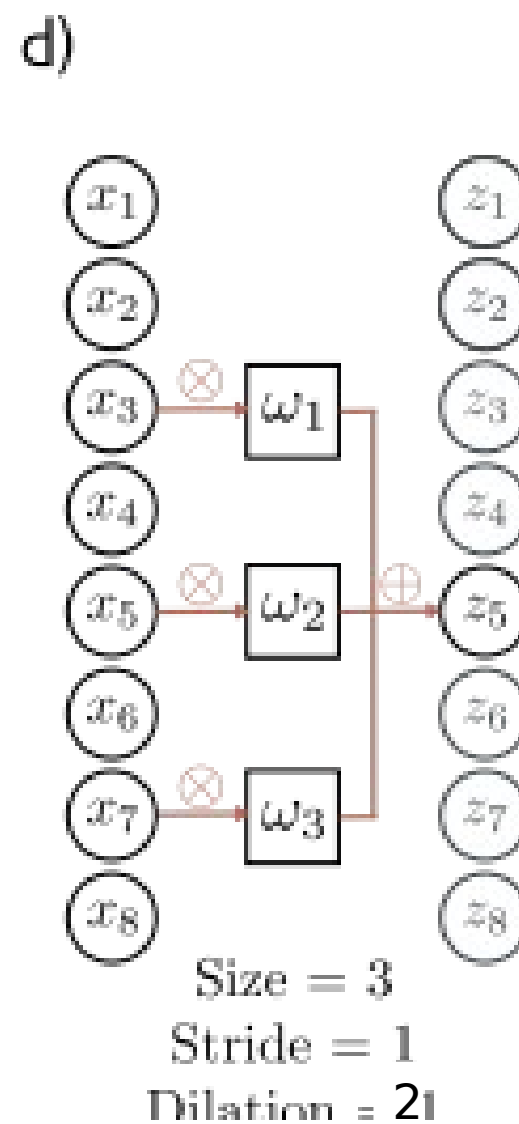
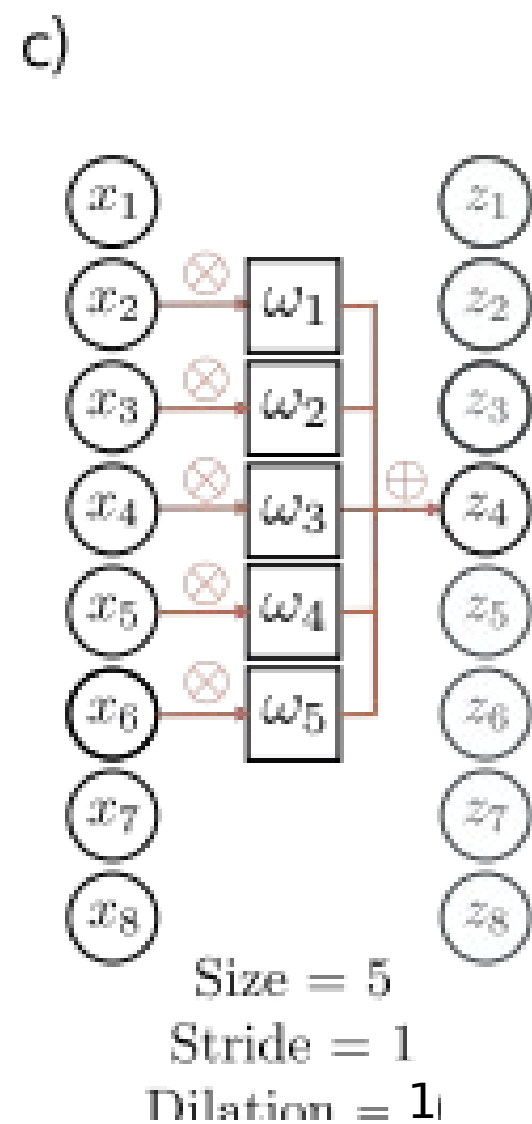
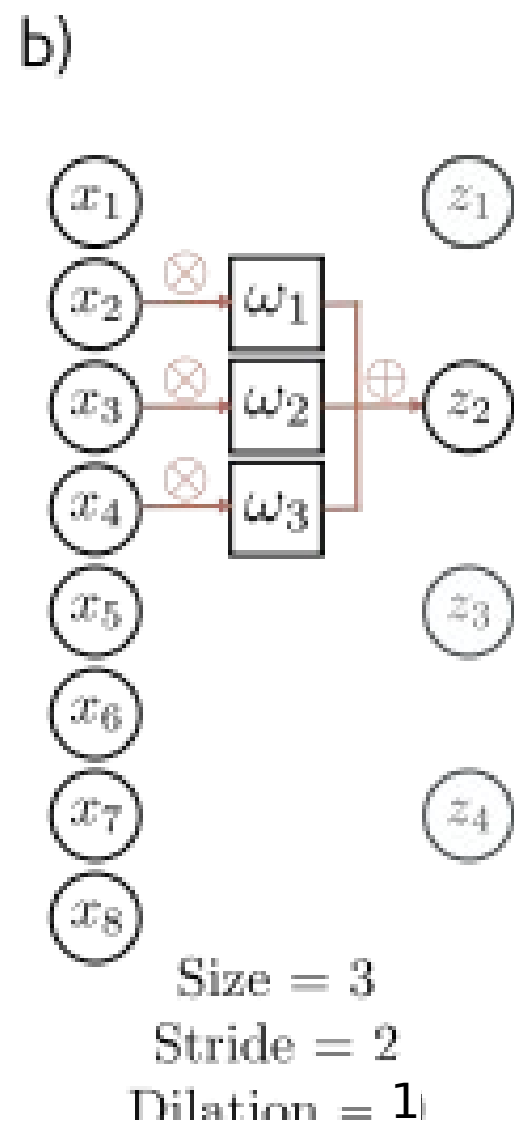
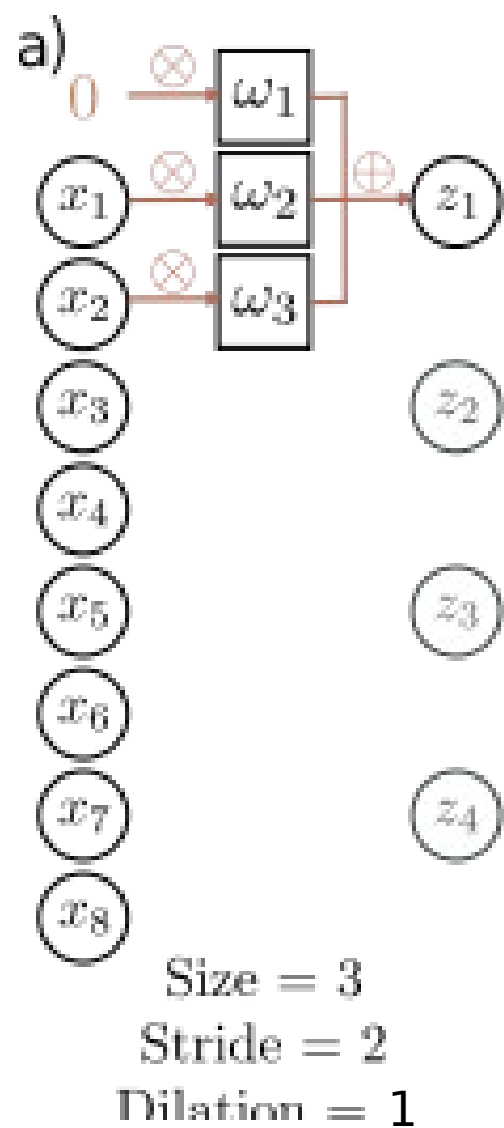
# Stride, kernel size, and dilation

- **Stride** = shift by  $k$  positions for each output
  - Decreases size of output relative to input
- **Kernel size** = weight a different number of inputs for each output
  - Combine information from a larger area
  - But kernel size 5 uses 5 parameters
- **Dilated** or **atrous** convolutions = intersperse kernel values with zeros
  - Combine information from a larger area
  - Fewer parameters

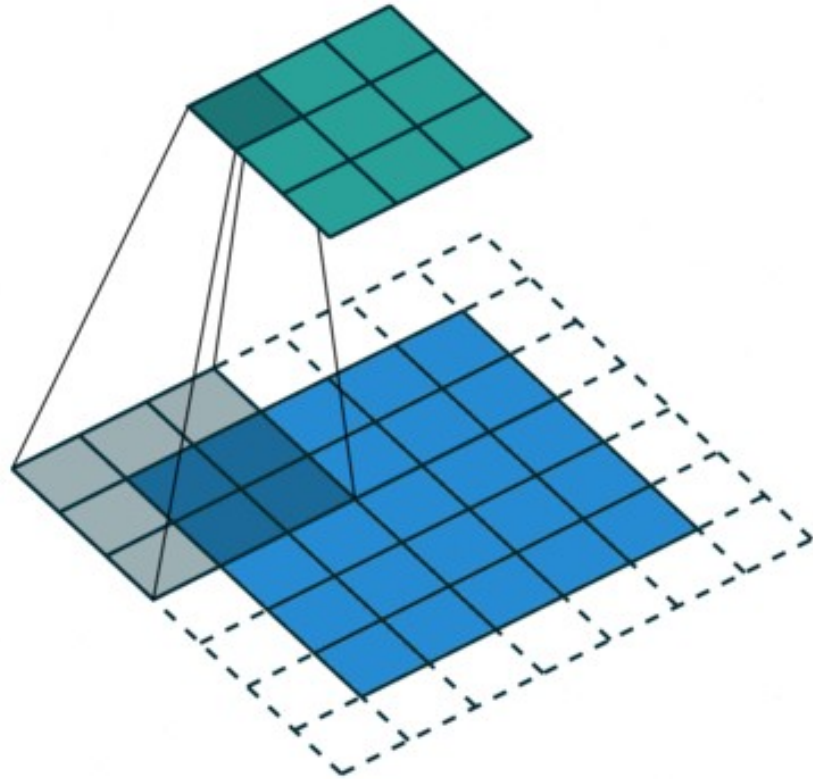




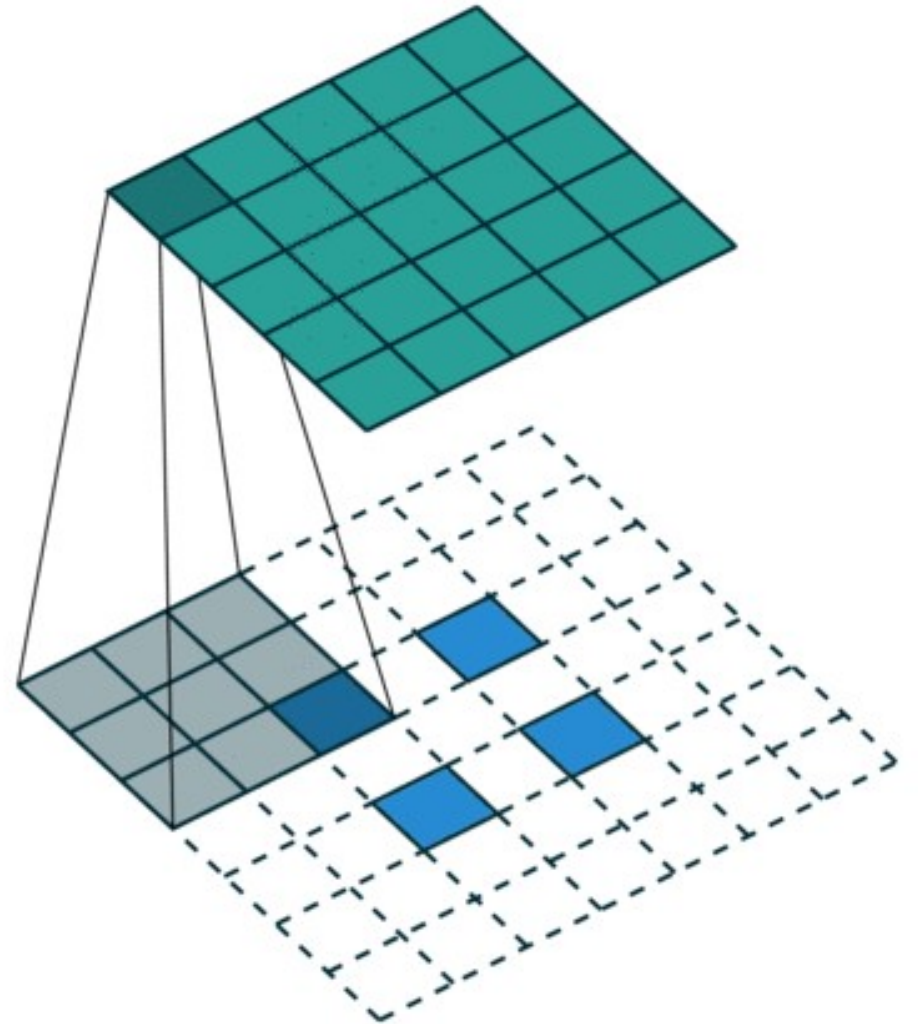




Strided convolution – apply to every other pixel – output is half as big



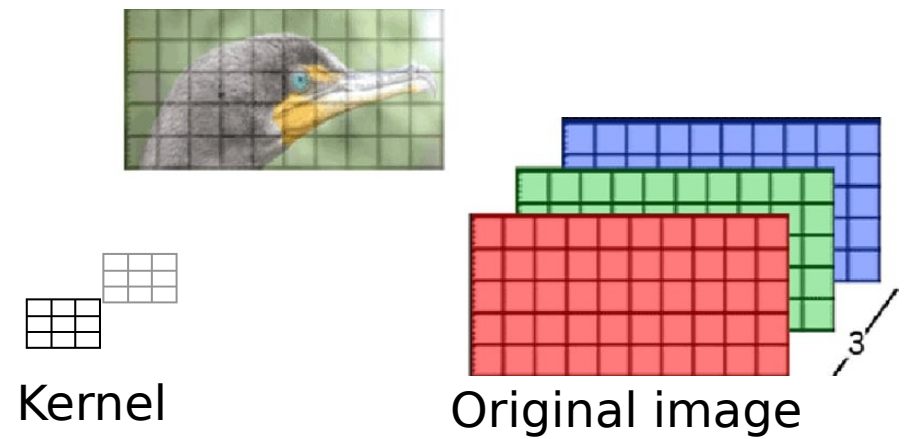
# Convolution to upscale an image



# Convolution equation

Output “image”

$$I'_{x,y,k} = \sum_{i=-K}^K \sum_{j=-K}^K \sum_{c=1}^C \underbrace{K_{i,j,c,k}}_{\text{red}} \cdot \underbrace{I_{x+i,y+j,c}}_{\text{purple}}$$



$I_{x,y,c}$  - value of input image at pixel location  $(x, y)$  and channel  $c$ .

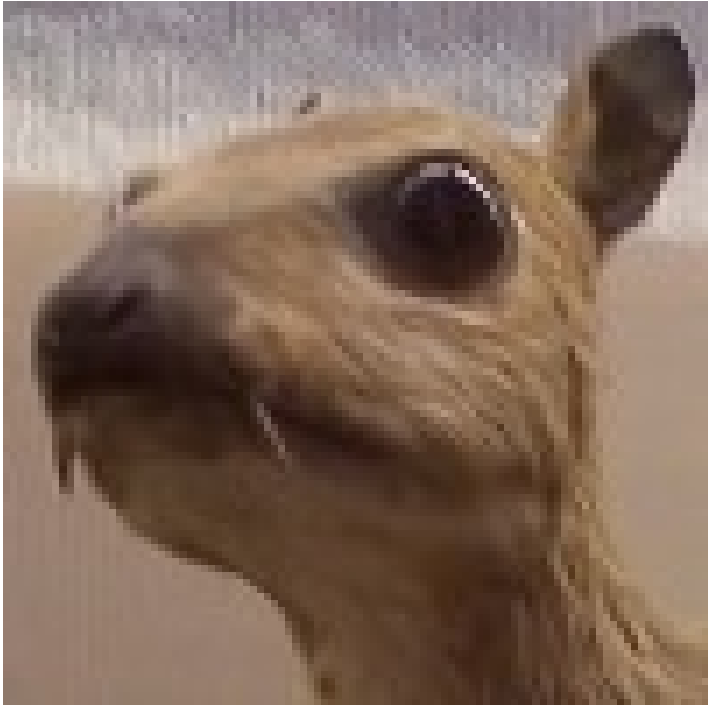
$I'$  is the output

$K_{i,j,c,k}$  is the weight of the convolution kernel at position  $(i, j)$  for input channel  $c$  and output channel  $k$ .

$C$  is the number of input channels.

$K$  is the size of the kernel (assuming a square kernel of size  $(2K + 1) \times (2K + 1)$ ).

# What do convolutions learn?



$$* \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} =$$



Weird example from [Wikipedia](#): lots of different image features are possible

What is the “receptive field” of a pixel on the right?

# What do convolutions at each layer learn?

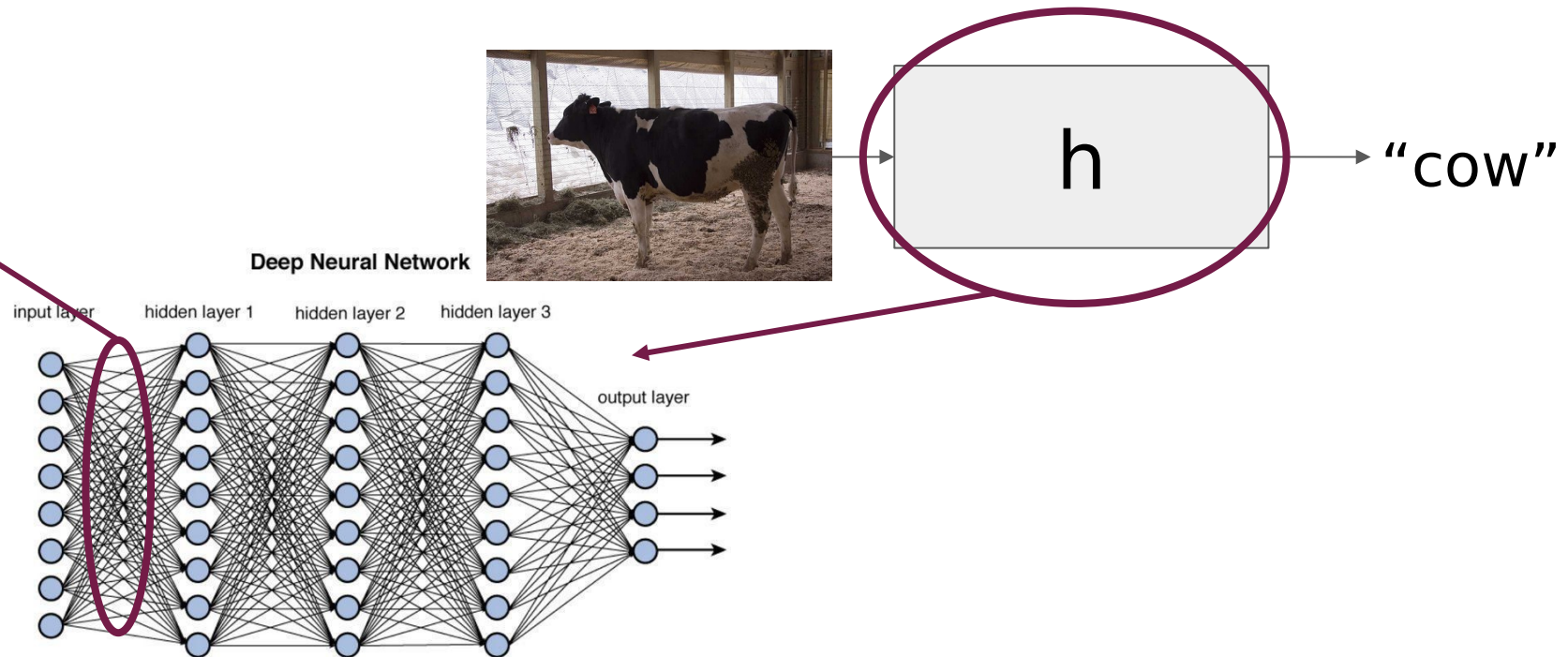
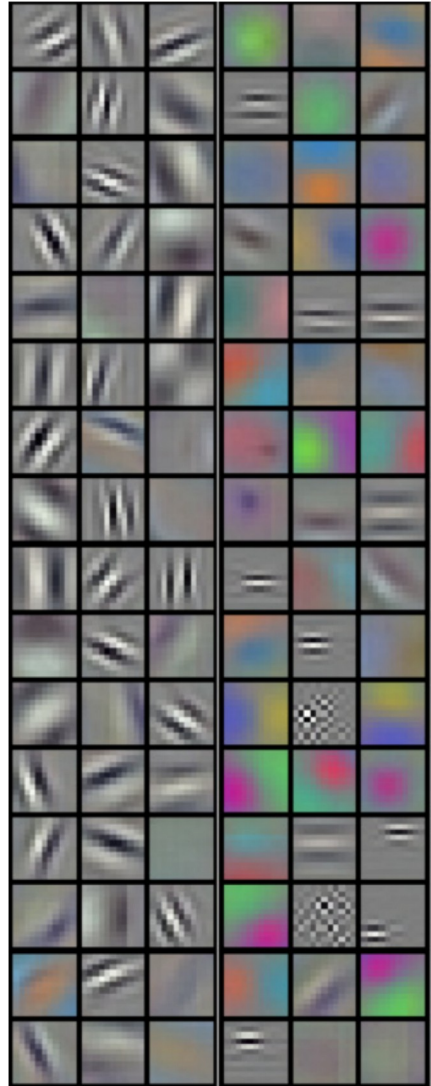
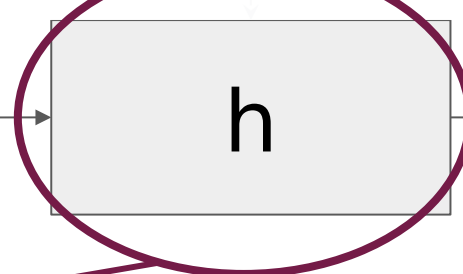
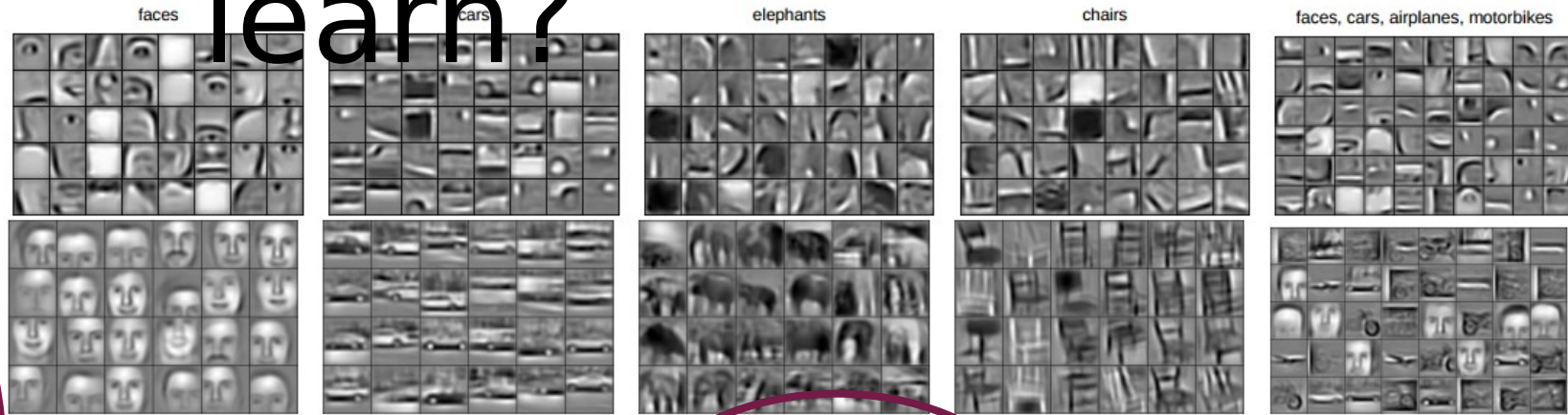
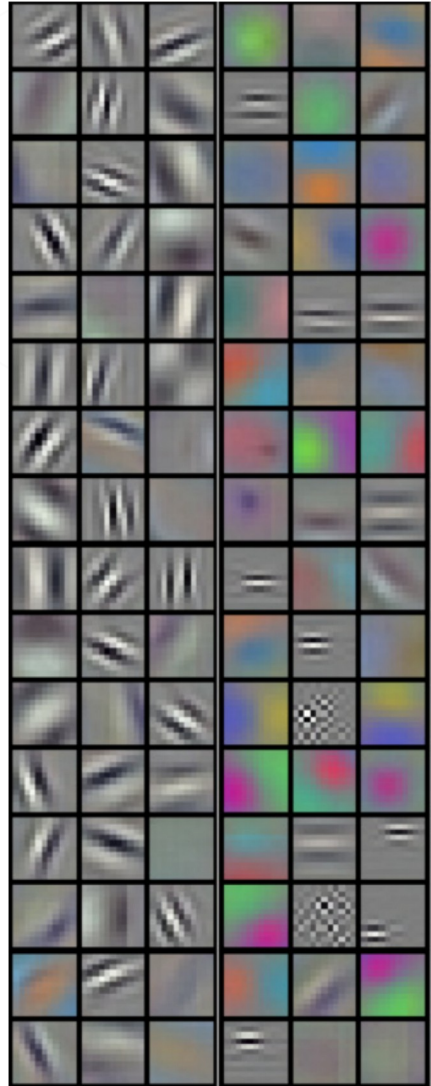


Figure 12.2 Deep network architecture with multiple layers.

# What do convolutions at each layer learn?



"cow"

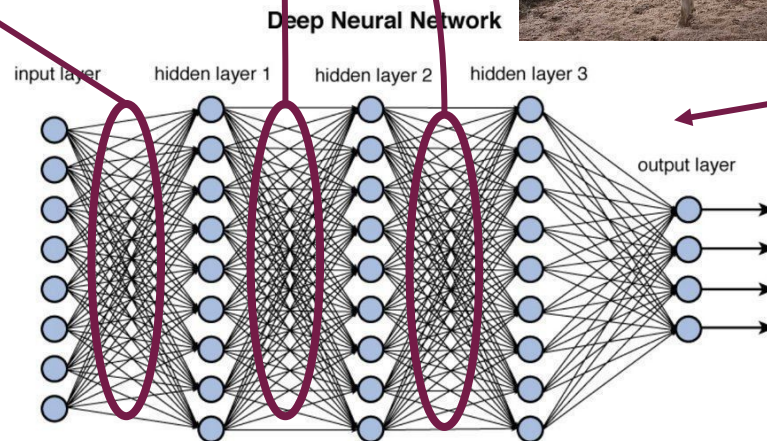
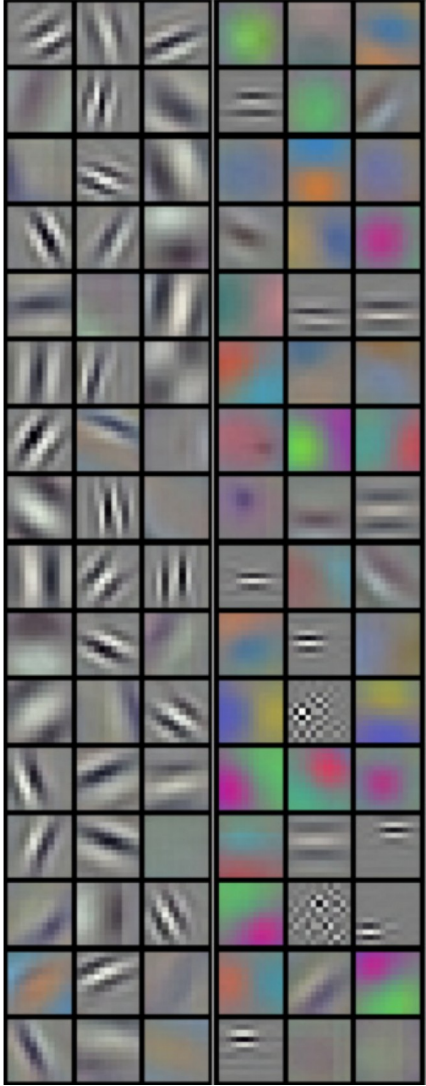


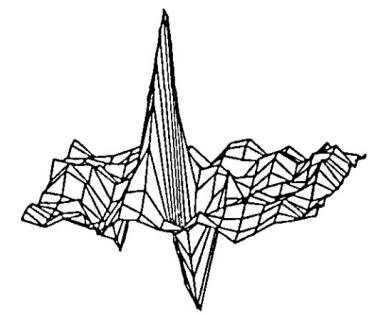
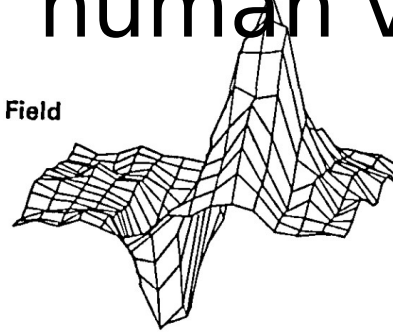
Figure 12.2 Deep network architecture with multiple layers.

# Are these familiar?

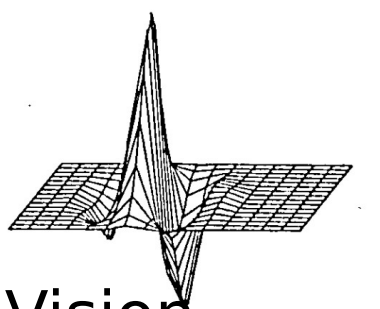
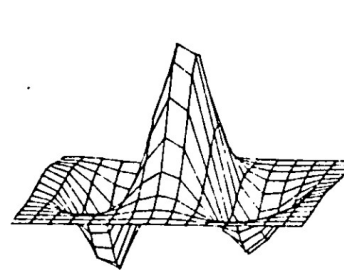
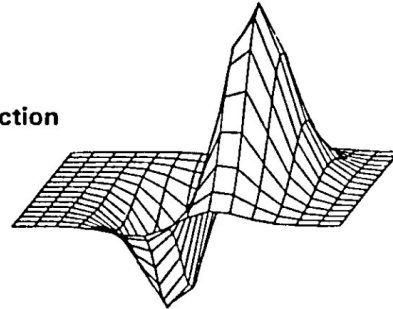
Neuroscientists discovered them in the 80s  
(called Gabor Filters) as fundamental to  
human vision



2D Receptive Field

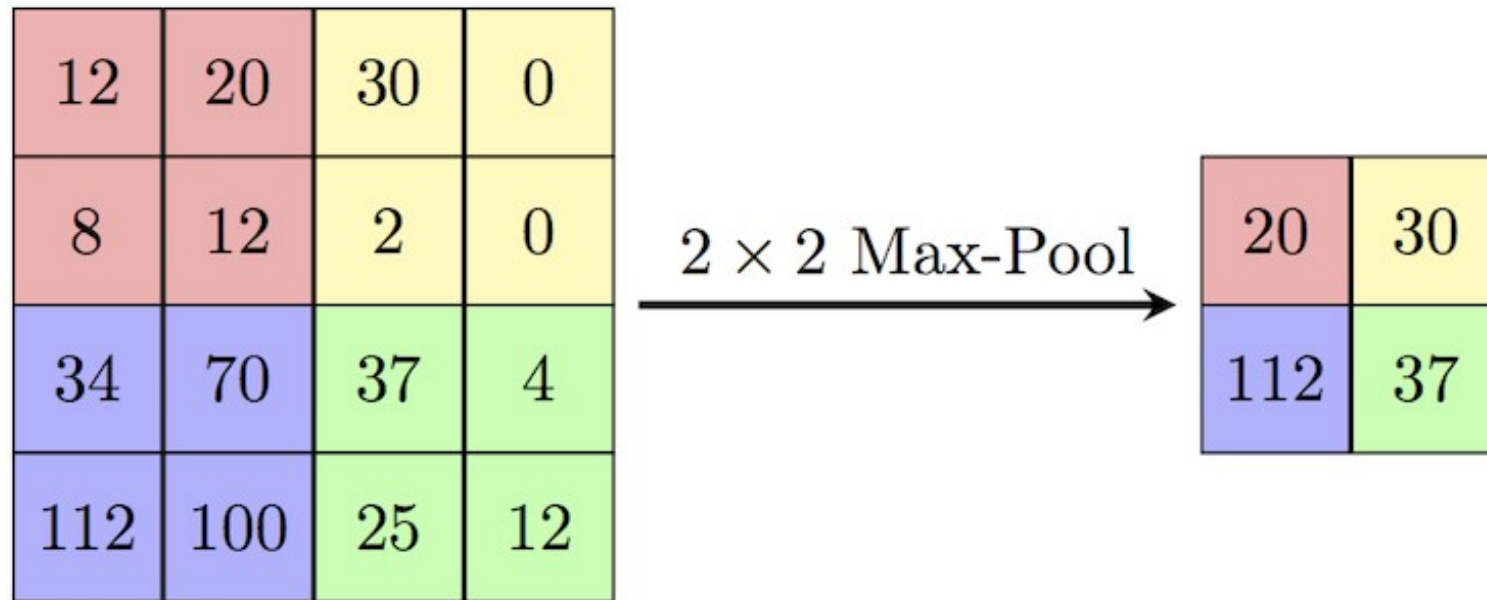


2D Gabor Function



Daugman, Vision  
Research, 1980

# Max pooling adds some robustness to translations



# Ways to reduce dimensionality of image

## **Max pooling**

- no parameters
- Simpler / cheaper operation
- Translation invariance

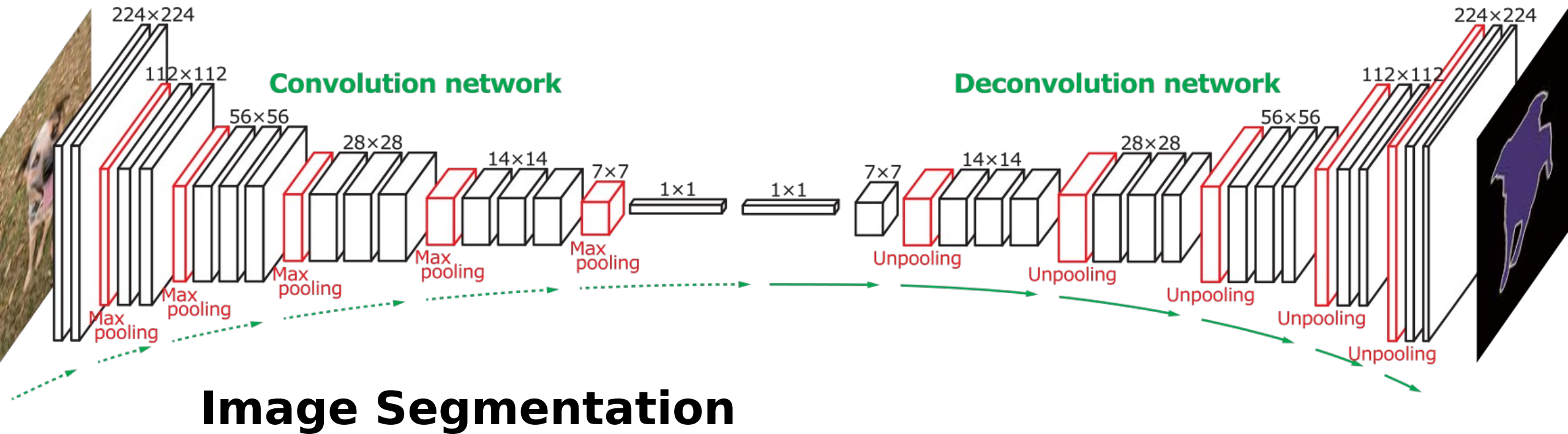
## **Strided convolutions**

- Learnable parameters
- More flexible

# Fully-Convolutional Networks (FCN)

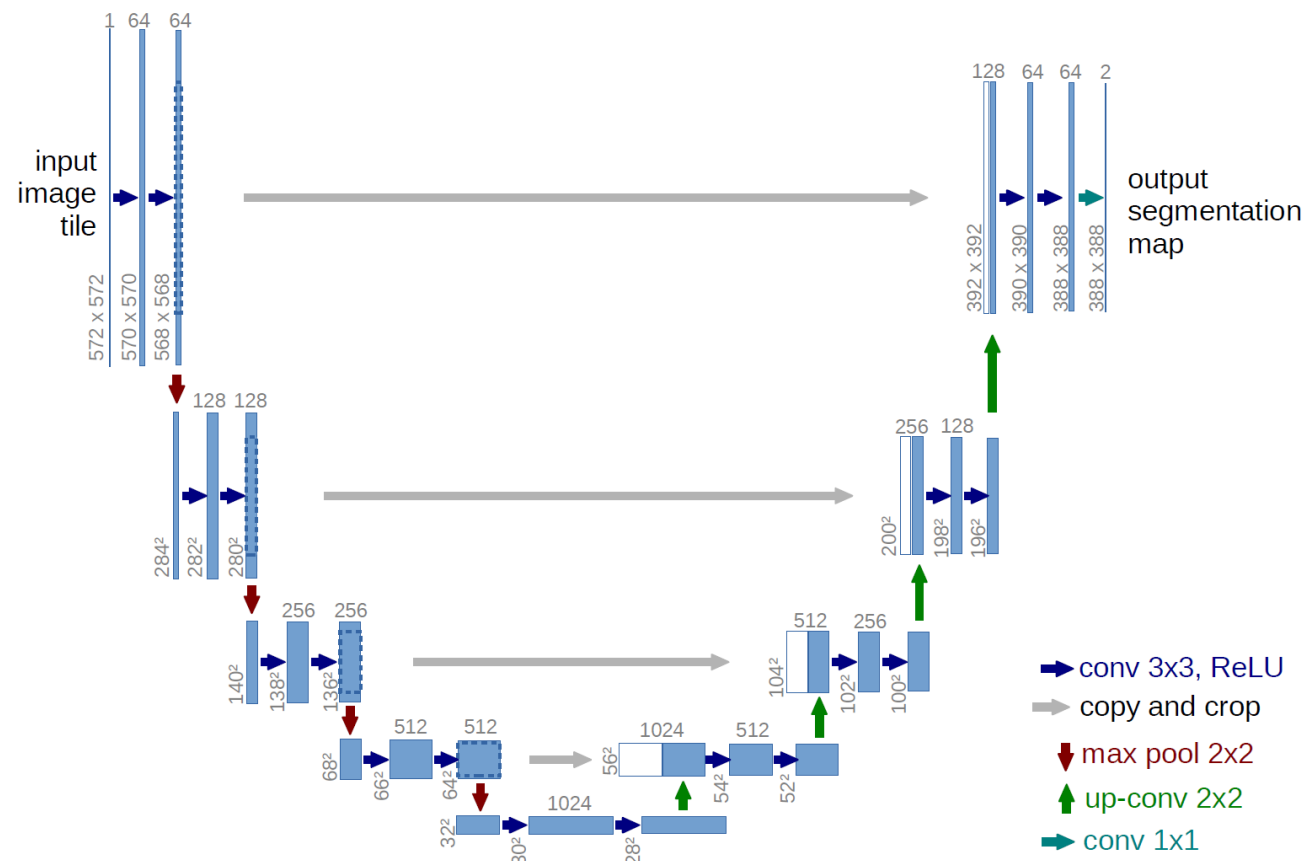
Uses Convolutions then deconvolutions

(<https://arxiv.org/abs/1411.4038>)



# U-Net

Like FCNs, but adds skip connections ([paper](#))

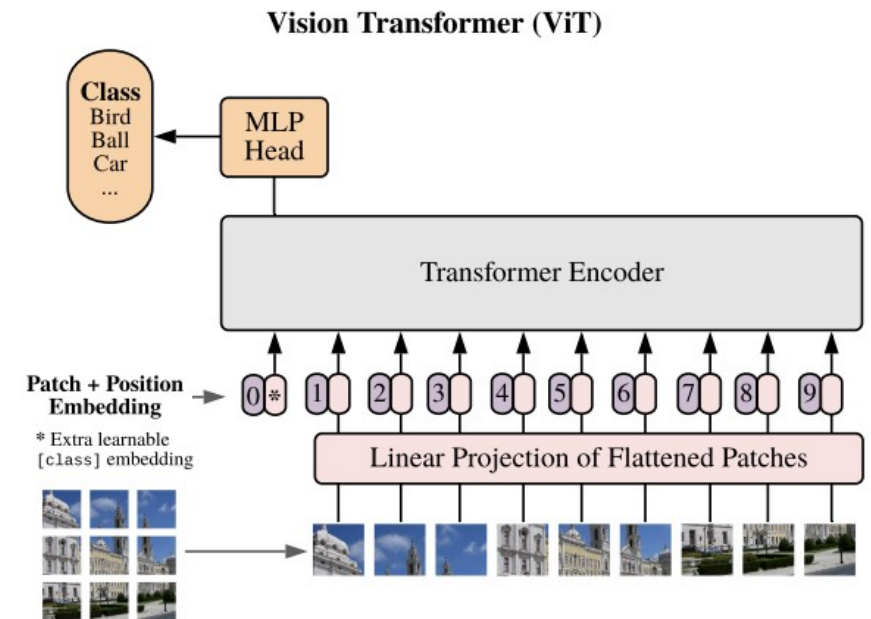


**Image Segmentation**

**Image generation (diffusion - later in course)**

# Abbreviated history of vision “backbones”

- Convolutional
  - VGG
  - Resnets, WideResNets
  - EfficientNet
- Vision transformer
  - Original
  - Swin transformer – hierarchical version ConvNext (early 2022)
- Combine conv + attention: MaxViT (late 2022)
- 2026? Attn more common, but arch less important than scale/training (MONDAY)



# Residual Neural Networks (ResNet)

Composed of Residual blocks.

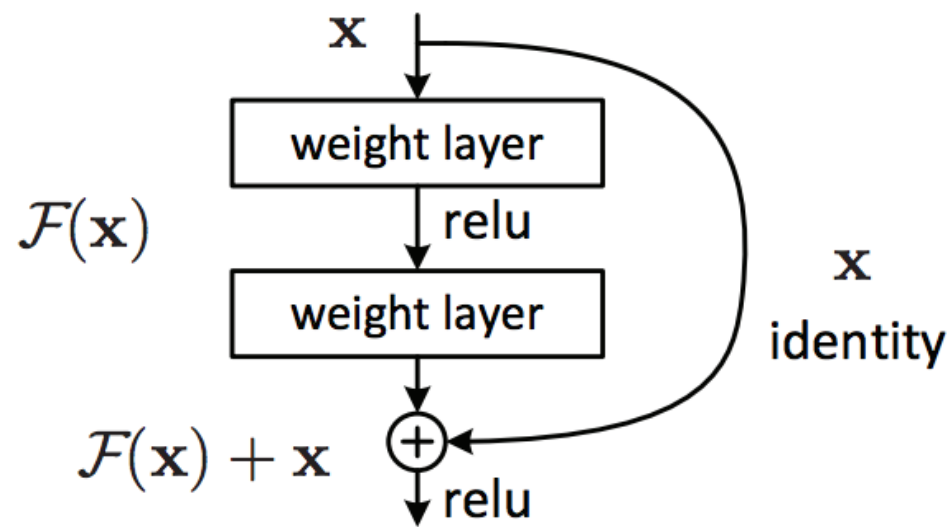
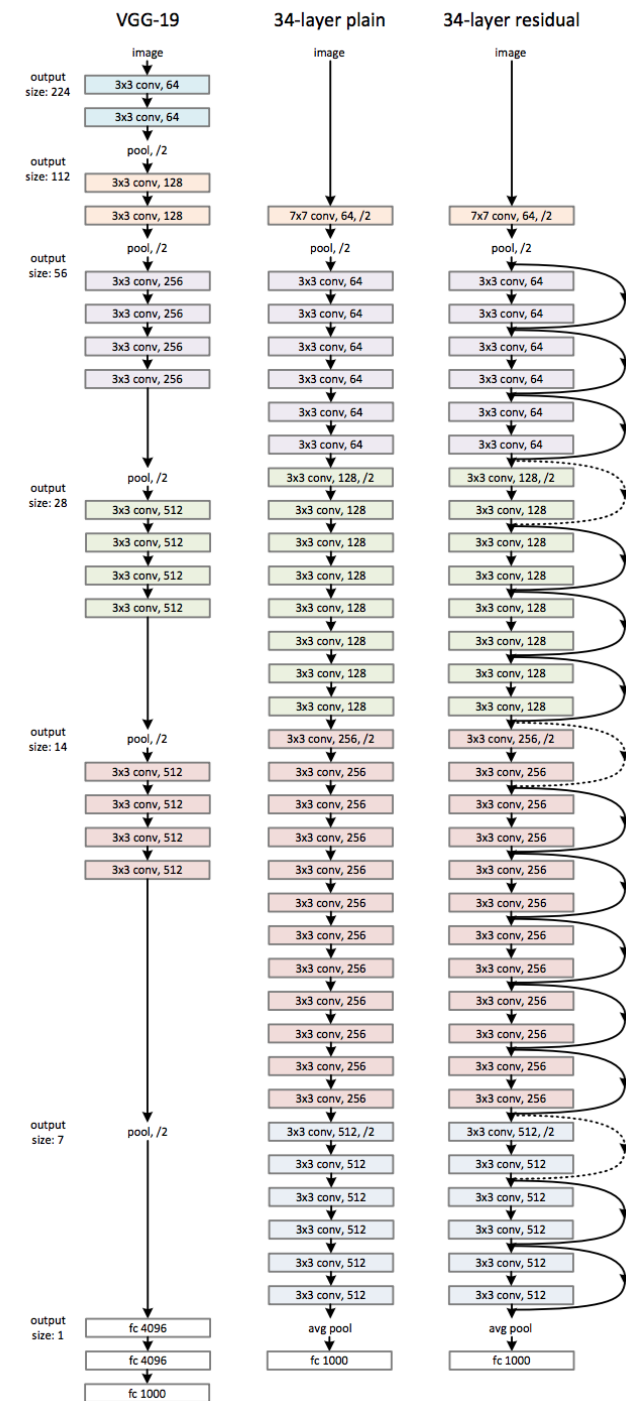


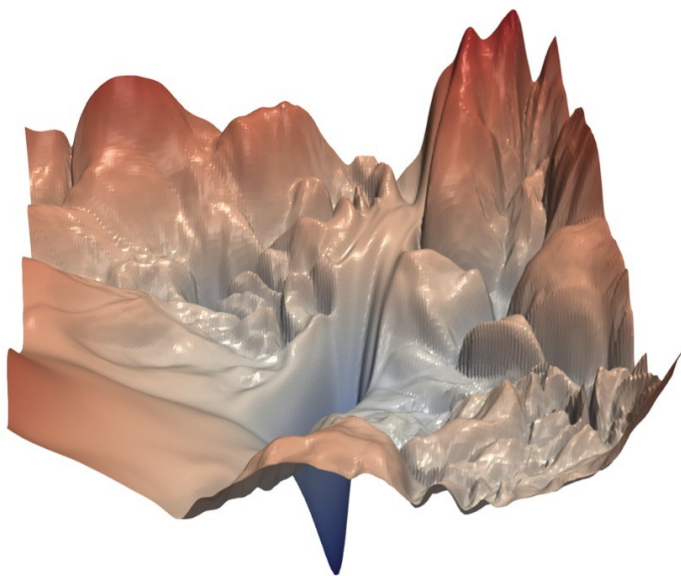
Figure 2. Residual learning: a building block.



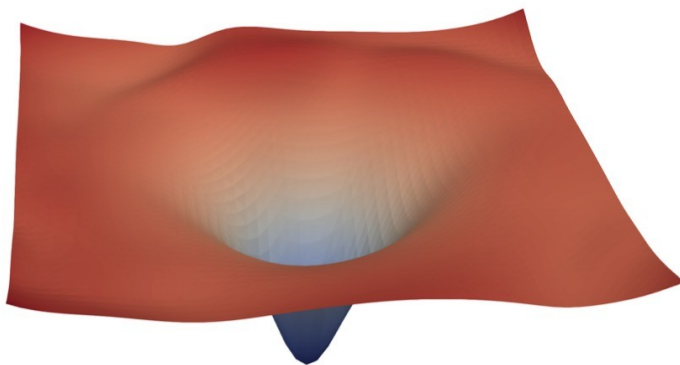
---

# Visualizing the Loss Landscape of Neural Nets

---



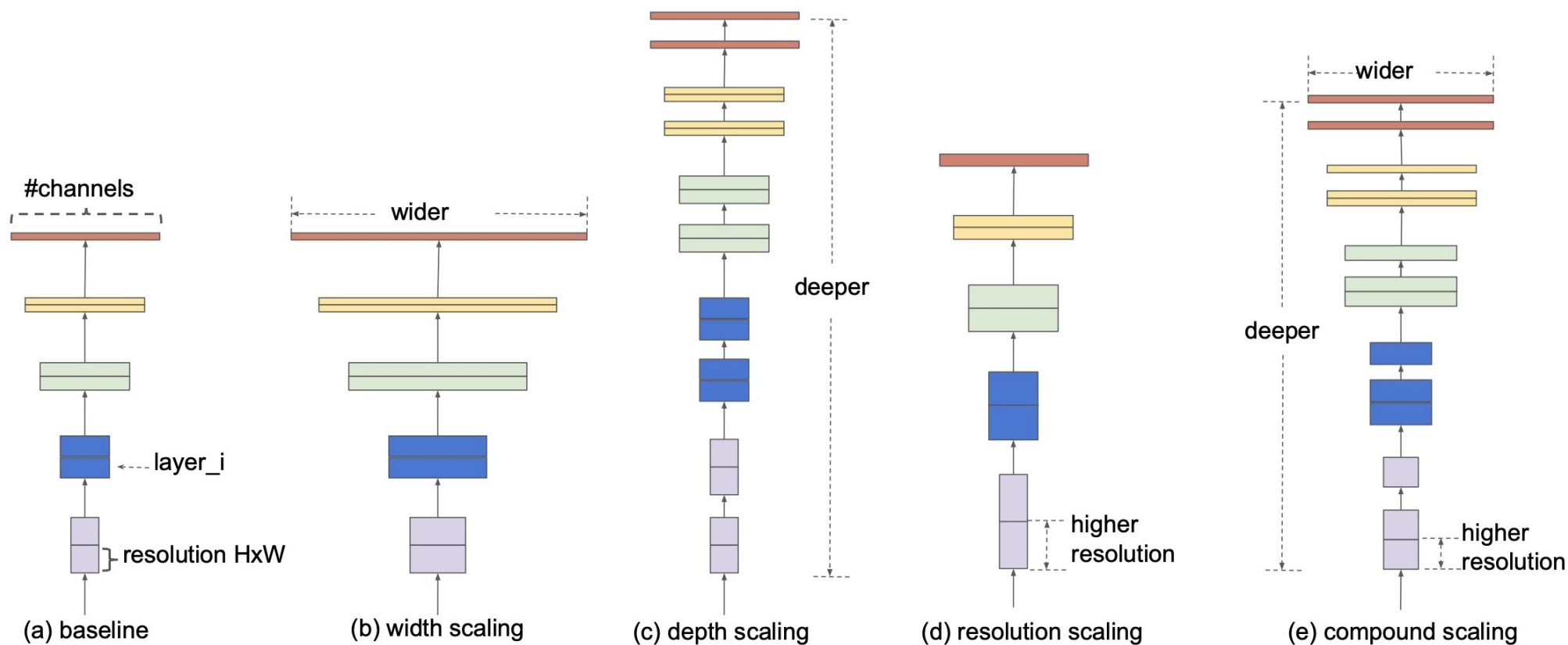
(a) without skip connections



(b) with skip connections

# How to grow your (residual) neural net

## EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks



**Figure 2. Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

# The Vision Transformer

Published as a conference paper at ICLR 2021

---

## AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

**Alexey Dosovitskiy<sup>\*,†</sup>, Lucas Beyer<sup>\*</sup>, Alexander Kolesnikov<sup>\*</sup>, Dirk Weissenborn<sup>\*</sup>,  
Xiaohua Zhai<sup>\*</sup>, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,  
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby<sup>\*,†</sup>**

<sup>\*</sup>equal technical contribution, <sup>†</sup>equal advising

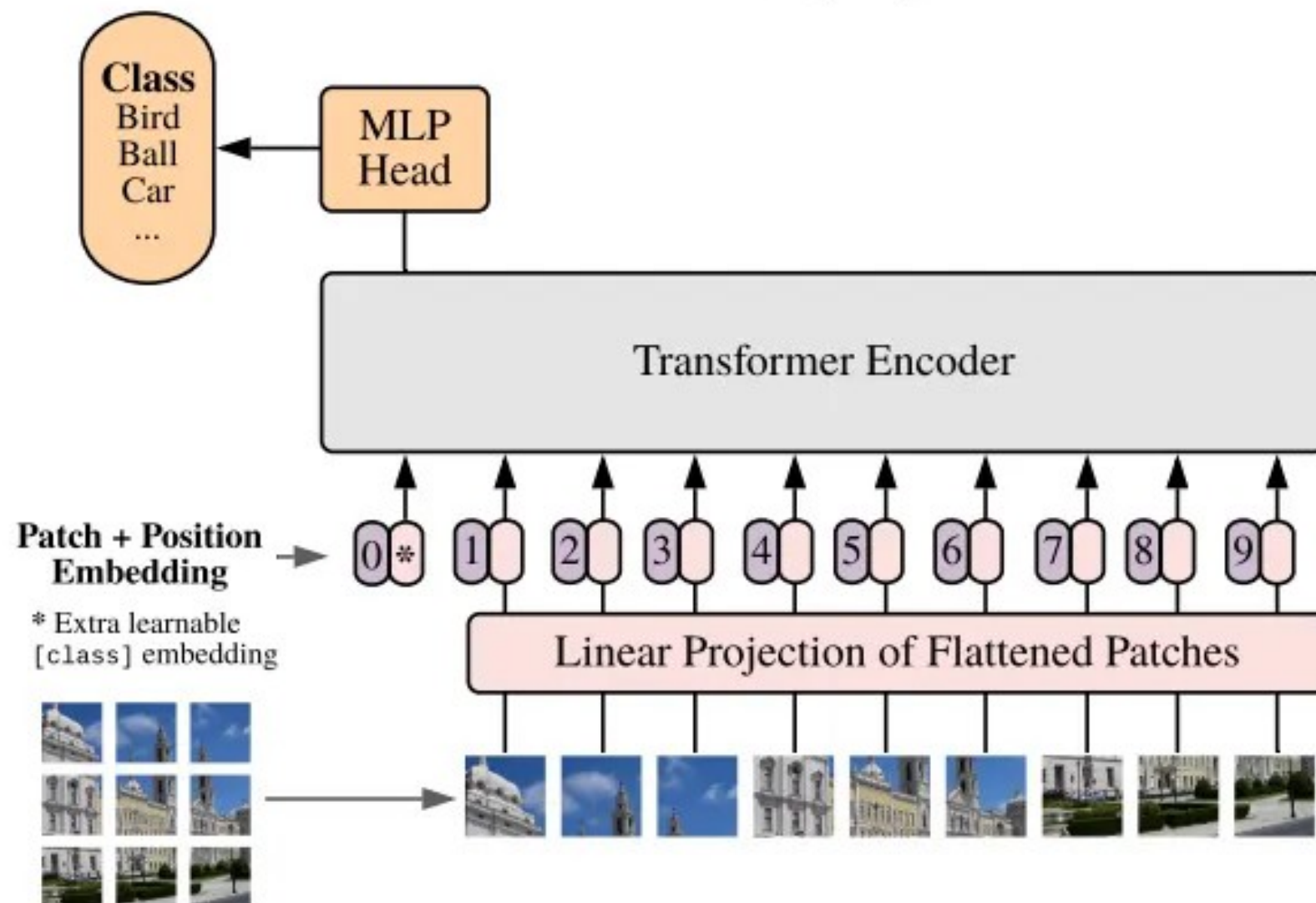
Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

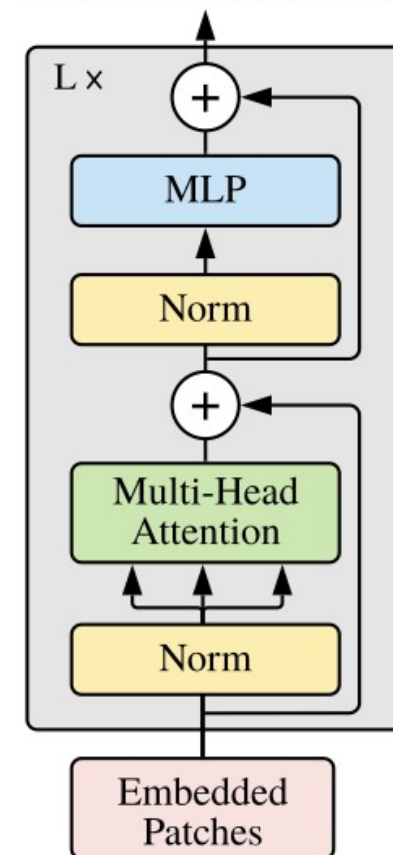
### ABSTRACT

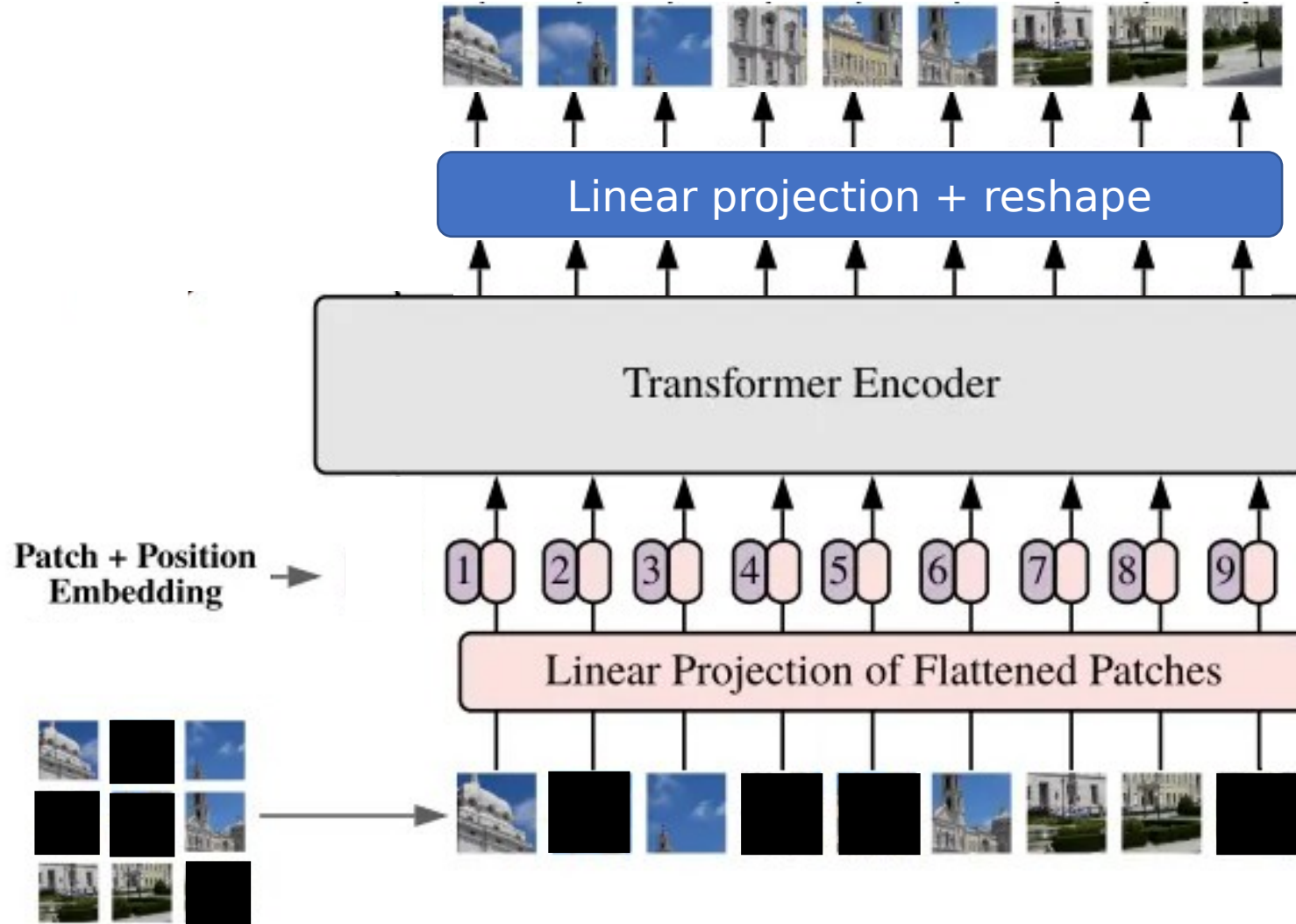
While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.<sup>1</sup>

# Vision Transformer (ViT)



## Transformer Encoder



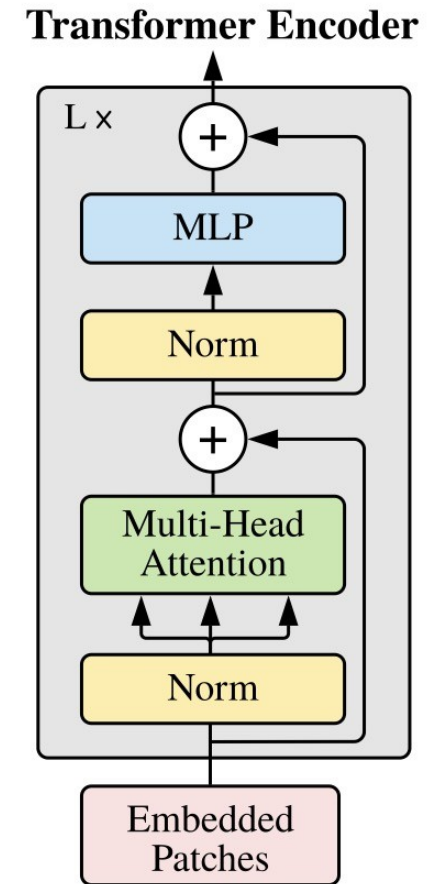
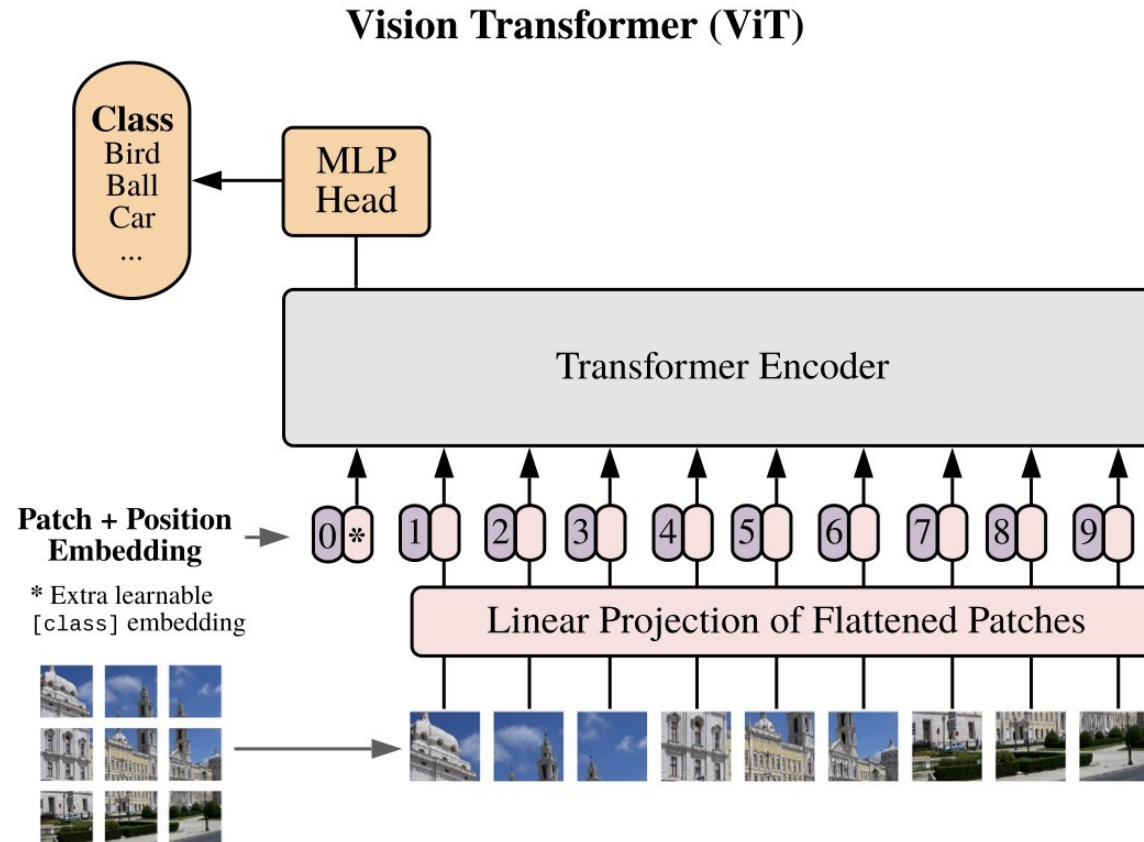


# Original vision transformer

Expensive  
compute and  
requires lots  
of training  
data

Swin transfor  
mer

was  
hierarchical /  
more efficient  
for large  
images



# Back to convolution again

Zhuang Liu<sup>1,2\*</sup> Hanzi Mao<sup>1</sup> Chao-Yuan Wu<sup>1</sup> Christoph Feichtenhofer<sup>1</sup> Trevor Darrell<sup>2</sup> Saining Xie<sup>1†</sup><sup>1</sup>Facebook AI Research (FAIR) <sup>2</sup>UC BerkeleyCode: <https://github.com/facebookresearch/ConvNeXt>

Pure architecture exploration paper, to find a convnet that competes with attention, using attention-inspired modifications:

- larger kernels, (transformers work on large patches)
- layer norm (like transformers),
- Inverted bottleneck layers,

...

ResNet Block

ConvNeXt Block

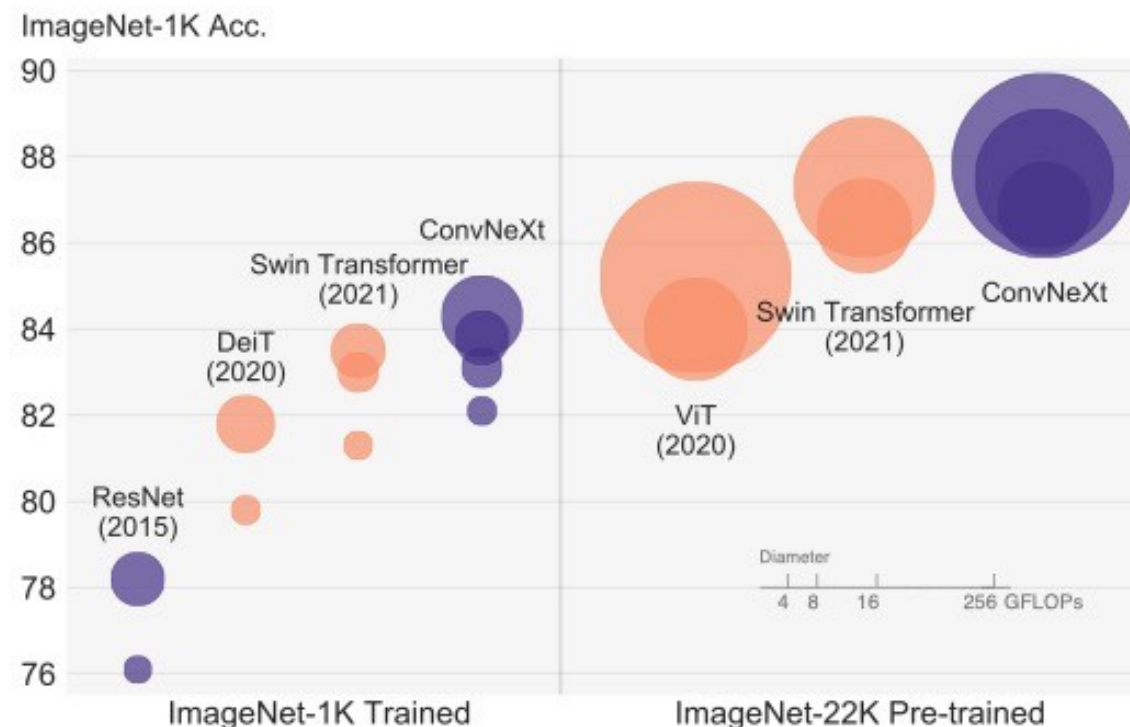
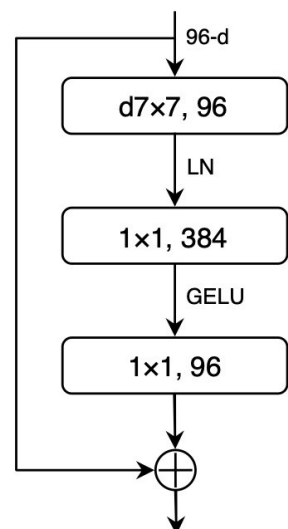
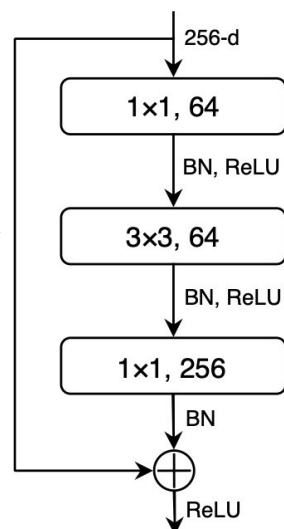
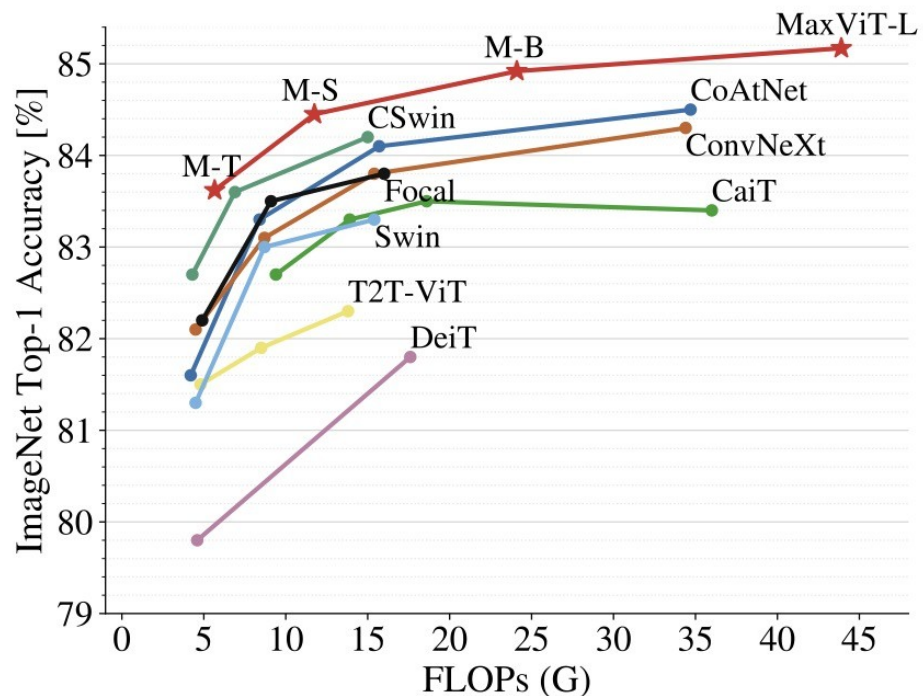
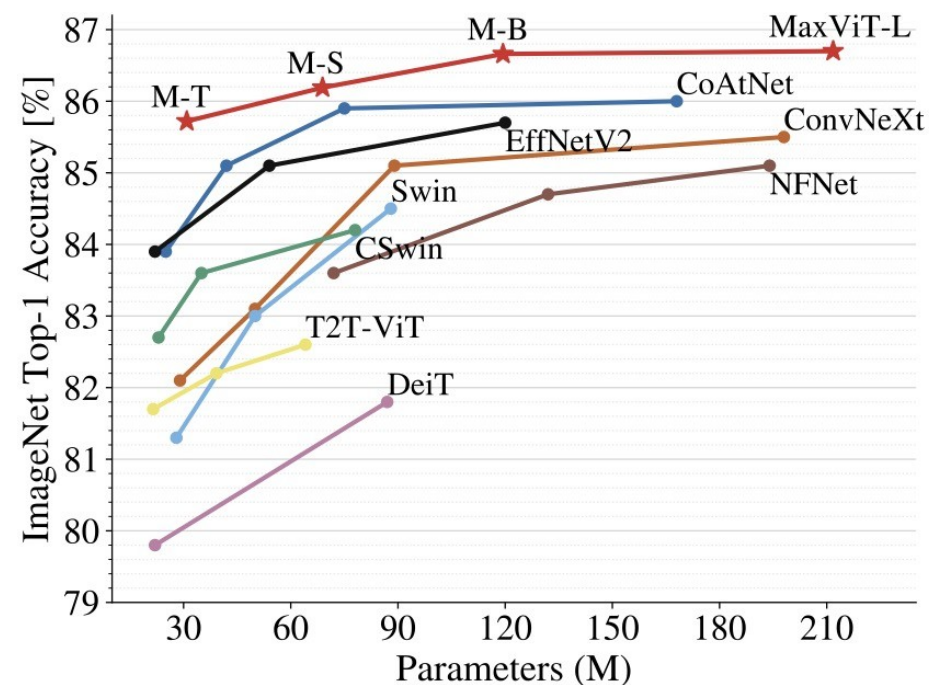


Figure 1. **ImageNet-1K classification** results for • ConvNets and ○ vision Transformers. Each bubble's area is proportional to FLOPs of a variant in a model family. ImageNet-1K/22K models here take  $224^2/384^2$  images respectively. We demonstrate that a standard ConvNet model can achieve the same level of scalability as hierarchical vision Transformers while being much simpler in design.

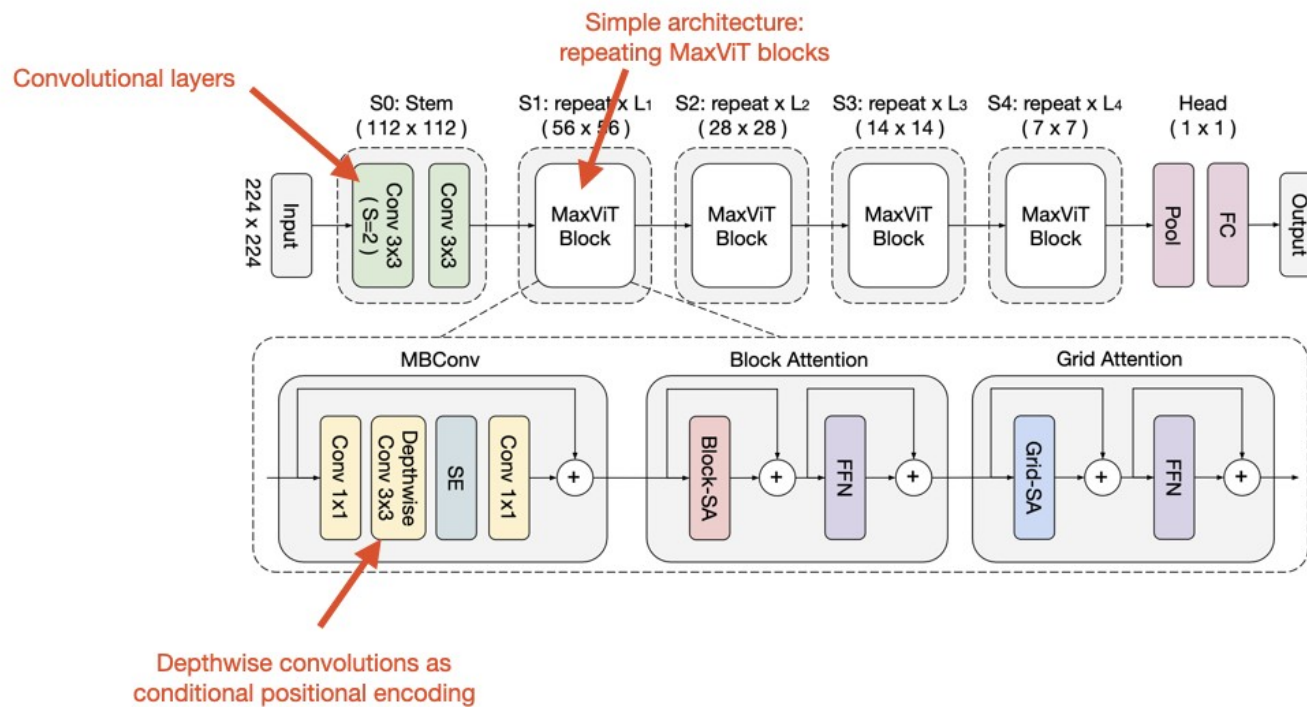
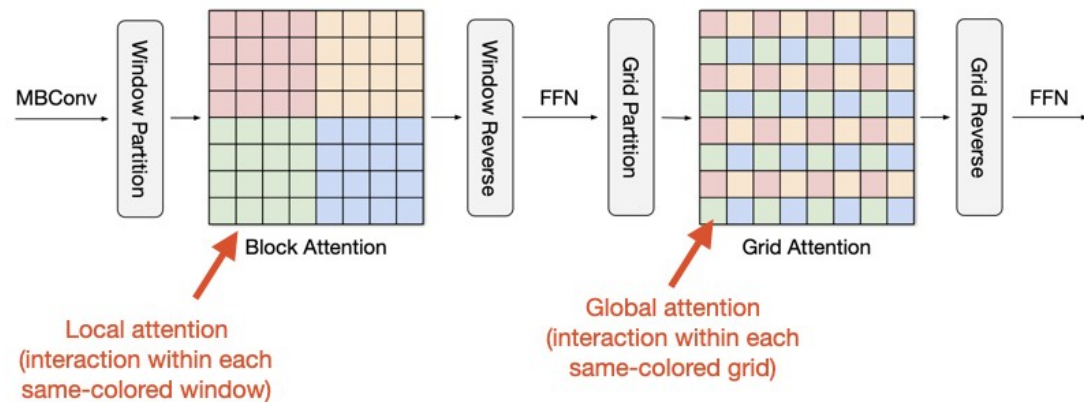
# Best of both worlds: MaxViT



(a) Accuracy *vs.* FLOPs performance scaling curve under ImageNet-1K training set at input resolution 224x224.



(b) Accuracy *vs.* Parameters scaling curve under ImageNet-1K fine-tuning setting allowing for higher sizes (384/512).



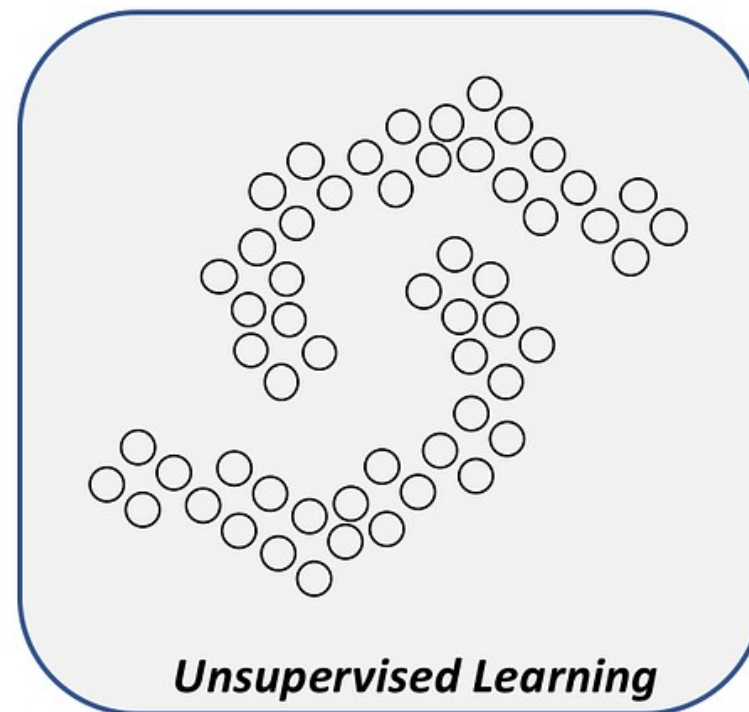
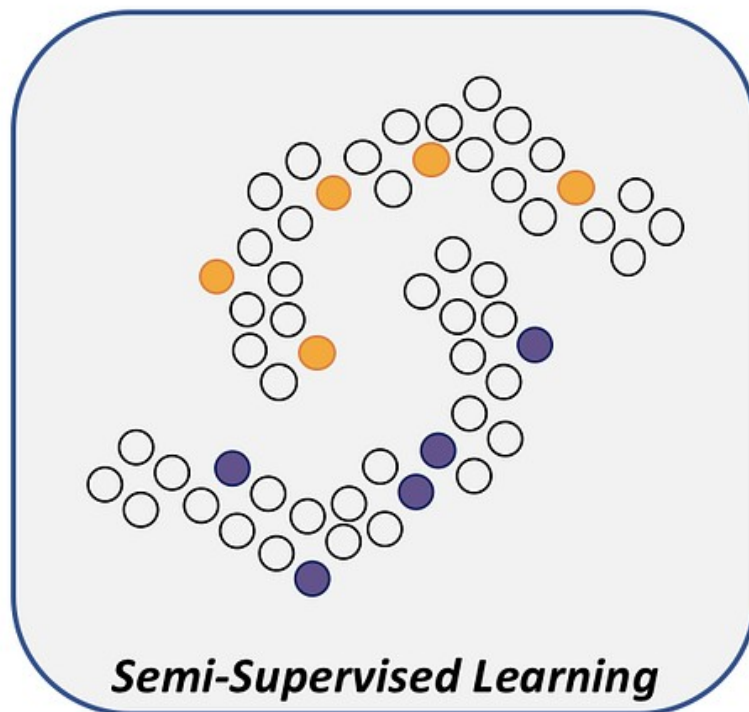
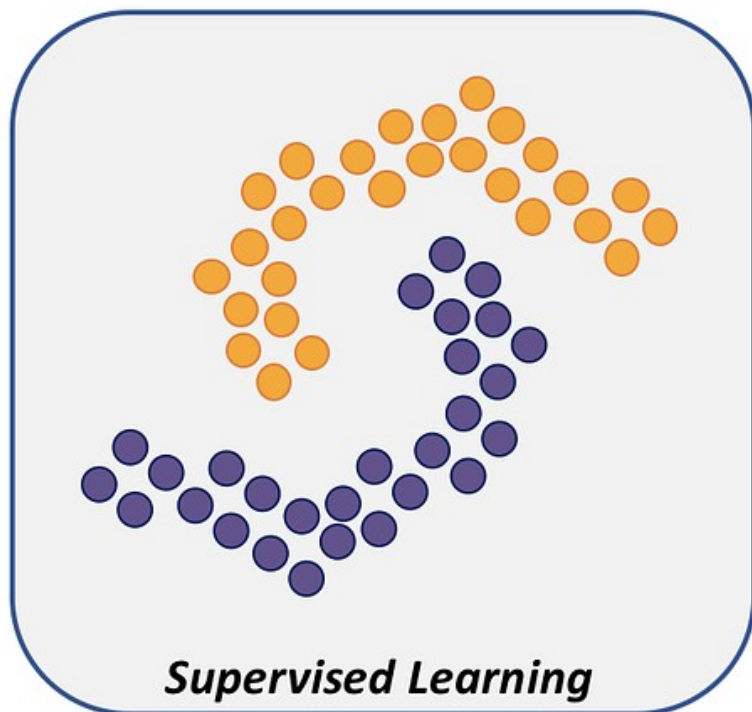
# Next time: semi-supervised and self-supervised / contrastive methods in vision

Contrastive style like CLIP/BLIP,

DINO

Multimodal

# Semi-supervised Vision



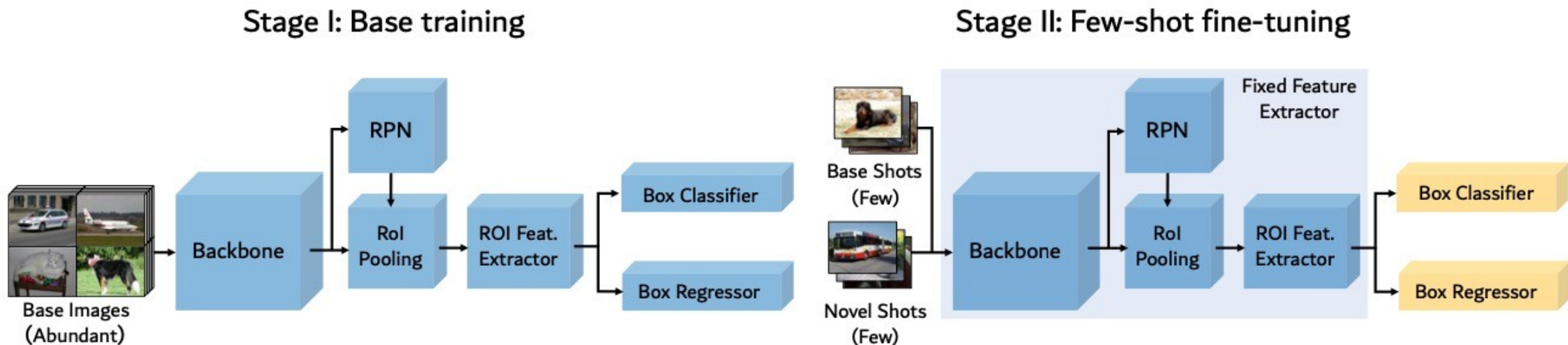
How to exploit the large number of unlabeled examples given in semi-supervised learning?

# Transfer learning – Always start here if possible

- Pre-train a good vision backbone on ImageNet  
(Will diffusion models become the “foundation” for all vision tasks? I think so – seeing evidence in the literature. Arguably CLIP models.)
- Adapt to your new task

# Frustratingly Simple Few-Shot Object Detection

Xin Wang<sup>\*1</sup> Thomas E. Huang<sup>\*2</sup> Trevor Darrell<sup>1</sup> Joseph E. Gonzalez<sup>1</sup> Fisher Yu<sup>1</sup>



*Figure 1.* Illustration of our two-stage fine-tuning approach (TFA). In the base training stage, the entire object detector, including both the feature extractor  $\mathcal{F}$  and the box predictor, are jointly trained on the base classes. In the few-shot fine-tuning stage, the feature extractor components are fixed and only the box predictor is fine-tuned on a balanced subset consisting of both the base and novel classes.

# A BASELINE FOR FEW-SHOT IMAGE CLASSIFICATION

Guneet S. Dhillon<sup>1</sup>, Pratik Chaudhari<sup>2\*</sup>, Avinash Ravichandran<sup>1</sup>, Stefano Soatto<sup>1,3</sup>

<sup>1</sup>Amazon Web Services, <sup>2</sup>University of Pennsylvania, <sup>3</sup>University of California, Los Angeles  
{guneetsd, ravinash, soattos}@amazon.com, pratikac@seas.upenn.edu

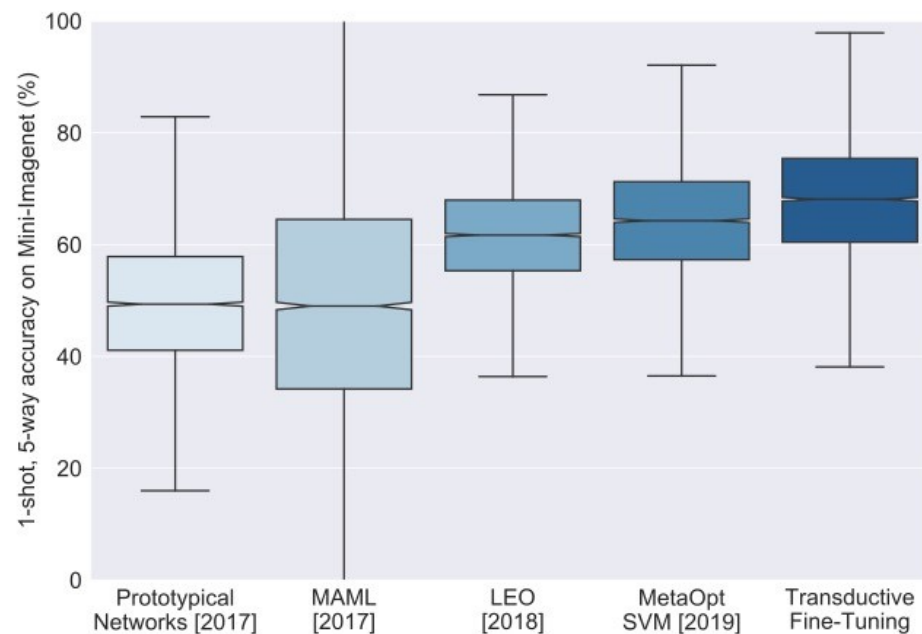


Figure 1: **Are we making progress?** The box-plot illustrates the performance of state-of-the-art few-shot algorithms on the Mini-ImageNet (Vinyals et al., 2016) dataset for the 1-shot 5-way protocol. The boxes show the  $\pm 25\%$  quantiles of the accuracy while the notches indicate the median and its 95% confidence interval. Whiskers denote the  $1.5\times$  interquartile range which captures 99.3% of the probability mass for a normal distribution. The spread of the box-plots are large, indicating that the standard deviations of the few-shot accuracies is large too. This suggests that progress may be illusory, especially considering that none outperform the simple transductive fine-tuning baseline discussed in this paper (rightmost).

## Pre-train, then fine-tune on target

### 3.2 TRANSDUCTIVE FINE-TUNING

In (4), we assumed that there is a single query sample. However, we can also process multiple query samples together, and perform the minimization over all unknown query labels. We introduce a regularizer, similar to Grandvalet & Bengio (2005), as we seek outputs with a peaked posterior, or low Shannon Entropy  $\mathbb{H}$ . So the transductive fine-tuning phase solves for

$$\Theta^* = \arg \min_{\Theta} \frac{1}{N_s} \sum_{(x,y) \in \mathcal{D}_s} -\log p_{\Theta}(y|x) + \frac{1}{N_q} \sum_{(x,y) \in \mathcal{D}_q} \mathbb{H}(p_{\Theta}(\cdot|x)). \quad (8)$$

$$\frac{1}{N_q} \sum_{(x,y) \in \mathcal{D}_q} \mathbb{H}(p_{\Theta}(\cdot|x))$$

*Transductive* fine-tuning also tries to give low entropy to unlabeled examples